

Combating alert fatigue with AlertPro: Context-aware alert prioritization using reinforcement learning for multi-step attack detection

Xiaoyu Wang^{a,b}, Xiaobo Yang^a, Xueping Liang^c, Xiu Zhang^{a,b,*}, Wei Zhang^a, Xiaorui Gong^{a,b}

^a Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China

^b School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

^c Department of Information Systems and Business Analytics, Florida International University, Miami, FL, United States of America

ARTICLE INFO

Keywords:

Alert fatigue
Multi-step attack
Intrusion detection
Reinforcement learning
Expert feedback
Active learning

ABSTRACT

Alert fatigue problems can have serious consequences for the enterprise security. When analysts become overwhelmed by the sheer number of alerts, high-risk alerts may go unnoticed or receive delayed responses, exposing the organization to potential cyber threats or data breaches. While current research on alert triage primarily concentrates on reducing false positives, analysts still face a shortage of resources to investigate all true alerts. The key to resolving this issue lies in the prioritization of alerts based on their potential severity, allowing analysts to allocate their efforts effectively.

This paper introduces AlertPro, an alert prioritization framework that facilitates the alert triage and validation stage of typical SOC workflow. The AlertPro framework extracts context features from alert sequences and history features from alerts previously investigated by analysts, besides basic features from raw alert data. By presenting analysts with only the top-ranked potentially high-risk alerts in each query and continually updating these rankings based on feedback, AlertPro significantly streamlines the alert investigation process. To evaluate AlertPro, we conducted experiments on five datasets that are chosen or prepared specifically because they all include multi-step attacks. The results reveal that AlertPro is able to discover a previously undisclosed attack concealed within the public dataset iscx, illustrating its potential in enhancing security posture. We also evaluate the feature importance in anomaly detection and conclude that employing context features yields better performance over basic features. The paper also explores the effectiveness of incorporating history features in active learning, achieving an average improvement of 30% in attack discovery rates. The processing time of AlertPro for re-ranking and selecting high-risk alerts is within 0.5 seconds, indicating that AlertPro can effectively work in real-time scenarios. AlertPro is limited to only using partial feedback and can be improved by incorporating richer feedback from experts. Overall, AlertPro mitigates alert fatigue, enabling security analysts to concentrate their efforts on high-priority threats.

1. Introduction

The Security Operations Center (SOC) is responsible for ensuring the security of the enterprise network, with a specific focus on efficiently detecting and responding to Advanced Persistent Threat (APT) attacks. An illustrative example of APT attacks is the campaign of TA459¹ in 2017, operated by a financially oriented attack group. In this campaign, the attacker gained initial access by exploiting a vulnerability CVE-2017-0199 in Microsoft Word. The attacker's tactics involved delivering a malicious Microsoft Word document via phishing emails. Once the

victim opened the document, the download of the file *power.rtf* was triggered. This file is actually an HTML Application (HTA) file that could download a PowerShell script named *power.ps1*. Subsequently, this PowerShell script was leveraged to fetch and execute the ZeroT payload *cgi.exe*, which, in turn, fetched and installed the PlugX Remote Access Trojan (RAT). The RAT trojan effectively collected information about the compromised system and transmitted it back to the Command and Control (C&C) server using HTTP protocol. Throughout this multi-step attack process, various alerts were generated by intrusion detection software, including the Phishing Email Detection System, Network In-

* Corresponding author at: Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China.

E-mail address: zhangxiu@iie.ac.cn (X. Zhang).

¹ <https://www.proofpoint.com/us/threat-insight/post/apt-targets-financial-analysts>.

trusion Detection System (NIDS), and Endpoint Detection and Response (EDR). These alerts generated during such attacks consisted of a mixture of true positives, false positives, and duplicate alerts. A survey conducted by FireEye (2015) indicates that over half of these alerts are false positives, with more than one-third being redundant. This flood of alerts is collected in the SOC, and security analysts in SOC are responsible to investigate these alerts and determine whether the network is under attack.

Alert fatigue leads to the ineffectiveness of SOC operations. When analysts are constantly inundated with alerts, it becomes increasingly challenging for them to prioritize and respond to attack alerts in a timely manner. The data breach incident at Target (Riley et al., 2014) serves as an example of the impact of alert fatigue. Overwhelmed by the sheer volume of alerts, Target's SOC analysts failed to recognize crucial warnings from their anti-intrusion software, leading to the devastating data breach incident. The efficiency of analysts across various organizations can be hindered by alert fatigue. A survey (Ponemon, 2015) indicates that organizations receive nearly 17,000 malware alerts per week, of which only 19 percent are deemed reliable, and limited analytical capabilities allow only 4 percent of those alerts to be adequately analyzed. According to the survey, the estimated economic value of lost time due to false negatives or false positives is approximately 1.27 million dollars annually. These figures underscore the pressing need for addressing alert fatigue in SOC environments.

To tackle the issue of alert fatigue, several automated alert triage techniques have been proposed. These techniques, such as alert reduction, prioritization, and correlation approaches, aim to alleviate the burden on analysts by reducing the total number of alerts, prioritizing the most critical ones, and identifying relationships among different alerts. **Alert reduction** (Al-Saedi et al., 2012; Njogu and Jiawei, 2010) is a process aimed at reducing the number of alerts, particularly those that are redundant due to overlapping detection capabilities of various products. One common method involves clustering similar alerts, with each class serving as a meta-alert. **Alert prioritization** (Ban et al., 2021; Aminanto et al., 2019; van Ede et al., 2022; Liu et al., 2022a; Veeramachaneni et al., 2016) is the process of categorizing and ranking security alerts. It involves assigning a priority or severity level to each alert, guiding security analysts on response efforts the allocation. The goal of alert prioritization is to ensure that the most critical alerts receive immediate attention and response from analysts, while lower-priority alerts can be addressed at a later time or potentially automated. By effectively prioritizing alerts, security teams can efficiently manage their workload, allocate resources, and focus on mitigating the most significant risks. **Alert correlation** (Ning et al., 2002; Shittu et al., 2014; Haas and Fischer, 2018; Husák and Kašpar, 2019; Nadeem et al., 2021; Hassan et al., 2019; Liu et al., 2022b) is a crucial process that involves identifying and grouping related security alerts to achieve a comprehensive understanding of potential security incidents or attacks. It involves analyzing multiple alerts from various sources and systems to determine if they are interconnected or part of a larger, coordinated attack. By correlating alerts, security teams can uncover the broader context of an incident, identify patterns, and integrate disparate pieces of information to respond effectively to complex threats. This approach enhances the SOC's ability to detect sophisticated and multi-step attacks, ultimately bolstering the organization's overall cybersecurity posture.

Drawing from the experiences of one of the authors, who worked as a triage specialist in the SOC and conducted hands-on investigations of alerts for six months, two critical insights into the alert validation process have been gained: (1) **alerts with more comprehensive attack context are more likely to be critical attacks**; (2) **adjusting alert triage results based on expert feedback can reduce subsequent false positives and false negatives**.

Based on these two insights and a thorough review of existing research, two research gaps are identified. First, current methods for analyzing alerts primarily concentrate on the alerts themselves, often overlooking the broader attack context. This is particularly problem-

atic when considering multi-step attacks (Navarro et al., 2018a) or APT campaigns, which have contextual correlations between attack steps and are difficult to evaluate based solely on the alert's attributes (Zhang and Ma, 2022). For example, an *AWVS Port Scan* alert might initially seem insignificant, but when accompanied by an *Exploit CVE-2020-14750 on Oracle Weblogic Server* alert, it becomes critical. Second, existing detection methods lack the capability to automatically update the model using expert feedback (Wang et al., 2020), thereby hindering the improvement of assessment accuracy and the prevention of future similar mistakes. Although alert triage models provide recommendations for the priority of alerts, analysts are still required to review the results and rectify any false positives or false negatives. Additionally, there is a persistent occurrence of incorrect alerts, necessitating frequent manual correction and significantly increasing the workload. This emphasizes the need for a feedback method that can effectively address these limitations.

Challenge. In summary, the challenges associated with alert ranking determination are as follows:

- * Despite extensive research on individual alerts, current studies have overlooked the contextual semantics associated with them.
- * The existing alert triage methods are not suitable for alert analysis because they fail to leverage the implicit information provided by analysts' feedback to automatically update rank results.

In this paper, we propose AlertPro, a context-sensitive and self-evolving framework to automatically learn from human experts' experience to distinguish critical alerts from non-attack or low-risk alerts. Specifically, our framework consists of two modules: a context inference module which extracts basic and context features of alerts, and a self-evolution module, which automatically updates the model to identify more critical alerts based on constant feedback.

Contribution. The contributions of this paper are the following:

- * We model the alert analysis task as a human-in-the-loop process, and use context features from the alert sequences, and history features from security experts' feedback, besides extracting basic features of alerts.
- * We propose the AlertPro framework, which uses reinforcement learning to optimize the results by re-ranking the alerts using history features from expert feedback.
- * We evaluate the performance of AlertPro using 5 real-world datasets and the results demonstrate that both contextual feature selection and feedback-based active learning models can improve detection accuracy and efficiency.

This paper is organized as follows: Section 2 explains the alert process lifecycle in the SOC workflow. Section 3 summarizes related work. Section 4 outlines the architecture of AlertPro and details of context features and history features. Section 5 introduces the implementation of the AlertPro framework, a case study of how AlertPro can discover undisclosed attacks, and evaluates the performance of AlertPro, based on three research questions. We discuss the limitations and future work of AlertPro in Section 6 and conclude in Section 7.

2. Background of alert process lifecycle

In this section, we aim to provide the essential context necessary for a comprehensive understanding of the alert lifecycle and its relevance within the context of a Security Operations Center (SOC). Section 2.1 introduces the typical SOC workflow, highlighting AlertPro facilitates the alert triage and validation stage. We introduce the background of using basic, context, and history features in Section 2.2, 2.3, and 2.4, respectively. Section 2.2 presents an alert example that security analysts routinely encounter within a SOC environment. Section 2.3 subsequently delves into the alert lifecycle, drawing our motivation for

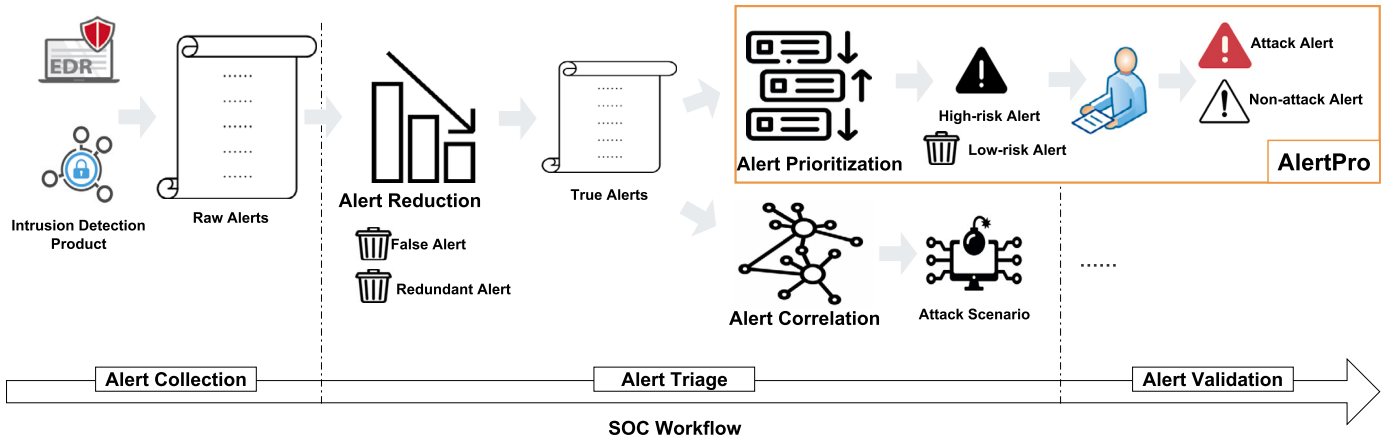


Fig. 1. SOC workflow. A typical SOC workflow consists of three stages: alert collection, triage, and validation. The alert collection stage collects raw alerts from various intrusion detection products. The alert triage techniques include three main branches: reduction, prioritization, and correlation. In the alert validation stage, analysts investigate and determine whether the alert is an attack alert. In this paper, AlertPro is a framework used for prioritizing alerts across the stages of triage and validation.

this study - to extend the response window of analysts. Following this, Section 2.4 provides deeper insight into the process of alert validation undertaken by analysts. Section 2.5 ultimately points out the core problem statement.

2.1. SOC workflow

Fig. 1 shows a typical SOC workflow, which involves 3 key analysis stages:

Alert collection refers to automatically gathering the alerts that are generated by the Intrusion Detection Product. These alerts, often referred to as raw alerts, include anomalies, rule violations, and suspicious behaviors.

Alert triage includes techniques such as reduction, prioritization, and correlation. Reduction involves filtering out false alerts or clustering redundant alerts. Prioritization assigns risk scores to alerts based on their severity or potential impact, identifying high-risk alerts. Correlation involves considering the context of surrounding alerts to identify patterns or relationships to discover attack scenarios. Alert triage is typically achieved by machine learning models. Existing research work mainly focuses on this stage, aiming to reduce false positives.

Alert validation requires experts' involvement, specifically triage experts, who investigate alerts from the alert triage stage and determine whether the alert is an attack alert. Details will be introduced in Section 2.4. This stage also includes reporting attack events and executing response actions, which are beyond the scope of this study.

In this paper, AlertPro is a framework used for prioritizing alerts during the stages of triage and validation. AlertPro combines the idea of human-in-the-loop (HITL) and aims to improve the performance of the machine learning model by taking advantage of experts' feedback. Specifically, analysts validate and make necessary corrections to the anomaly detection predictions. Then AlertPro updates the history feature of alerts and reranks alerts using an active learning model. The overview of AlertPro is shown in Fig. 5 and explained in Section 4.1.

2.2. Alert example with basic features

An example of a Suricata² alert in JSON format is shown in Fig. 2. Alerts contain three parts of fields: (1) index of the database record, such as *flow_id*, *in_iface*. The *flow_id* indicates the number of the corresponding raw data, and *in_iface* indicates the NIC that captured the flow. (2) information about the activity, such as *timestamp*, source IP

```
{
  "timestamp": "2018-11-03T23:25:46.238886+0000",
  "flow_id": 892271640868162,
  "in_iface": "ens4",
  "event_type": "alert",
  "src_ip": "122.9.52.215",
  "src_port": 35720,
  "dest_ip": "10.10.20.124",
  "dest_port": 80,
  "proto": "HTTP",
  "signature_id": 2024196,
  "signature": "ET WEB_CLIENT HTA File containing
Wscript.Shell Call - Potential Office Exploit Attempt",
  "match_data": "Content-Type application/hta Wscript.Shell",
  "attack_stage": "Exploit",
  "severity": 2
}
```

Fig. 2. An example alert from Suricata. In this paper, we extract 4 basic features that are highlighted in the raw alert: *src_ip*, *dest_ip*, *signature*, *attack_stage*. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

(*src_ip*), source port (*src_port*), destination IP (*dest_ip*), destination port (*dest_port*), protocol (*proto*). The *timestamp* is the detection time of this alert, and the rest is the 5-tuple of the flow. (3) information related to detection, such as matched *signatures* and its *signature_id*, *attack_stage* in multi-step attacks, and estimated *severity*.

In this paper, AlertPro extracts 4 basic features from the 4 raw alert fields that can differentiate this alert from others, which are used as partial inputs for anomaly detection. These raw fields are *src_ip*, *dest_ip*, *signature*, *attack_stage* which are highlighted in yellow in the Fig. 2. These 4 basic features are listed in Table 2 and explained in Section 4.2.1.

2.3. Extending the response window with context features

For a visual explanation, referring to the software vulnerability life cycle and patch window, we draw the alert life cycle and response window, as shown in Fig. 3. There are 4 key time points: (T1) attack performed by an adversary, (T2) alert triggered by some IDSs or EDRs, (T3) alert validated by analysts who identify attacks and block them, (T4) damage caused, either network paralyzed or data breach, indicating a failed SOC workflow. Time span between (T3) and (T4) is the response window. Our motivation is to detect attacks (T3) as early as

² <https://suricata.io/>.

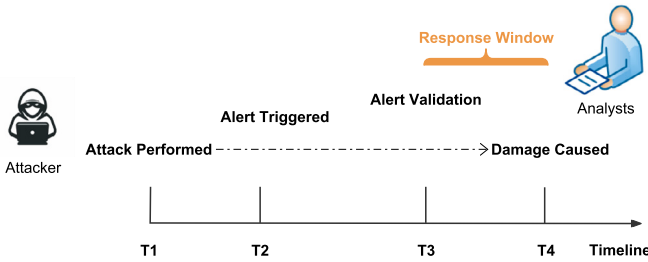


Fig. 3. Alert lifecycle and response window. Our main motivation is to detect attacks as early as possible and extend the alert response window of SOC.

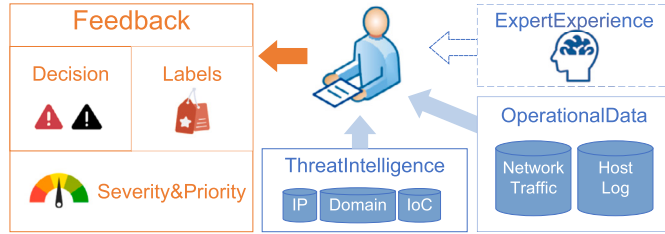


Fig. 4. Alert validation process. Based on expert experience, operational data, and threat intelligence, analysts decide whether the alert signifies an attack. They then assign labels and evaluate the severity and response priority accordingly.

possible, thereby expanding the alert emergency response window of SOC. At first glance, it may appear that alert analysts are combating alert fatigue. However, in reality, it is a strategic battle between alert analysts and attackers. The stealth, sophistication, and persistence of multi-step attacks often place analysts at a disadvantage. Consequently, our primary research motivation is to identify and prioritize alerts that are most likely to be associated with actual attacks. By doing so, we aim to optimize the allocation of analysts' limited time and effort, ensuring that they are not wasted on investigating non-attack alerts.

Compared with atomic attacks (such as SQL injection), multi-step attacks increase the difficulty of detection. Multi-step attack refers to a type of cyberattack that is planned and executed in multiple stages. Instead of launching a single attack, the attacker divides the attack into several steps, each relying on the success of the previous step. The time span of a multi-step attack is long, so the alerts belonging to the same attack are scattered in time. Therefore, in addition to investigating the alert itself, the alert sequence also deserves to be examined.

In this paper, in order to achieve a high attack discovery rate in multi-step attack scenarios, AlertPro extracts 7 context features from alert sequences, which are used as inputs for anomaly detection. The selection of these context features in this paper is primarily based on the experience of alert analysis and the characteristics of multi-step attacks. These 7 context features are listed in Table 2 and explained in Section 4.2.2.

2.4. Alert validation with history features

The alert validation process, as shown in Fig. 4, is a labor-intensive task. Even after the reduction and prioritization of the alert triage stage, there are still a large number of alerts that need to be manually investigated by analysts (Alahmadi et al., 2022). For each alert, the analyst investigates the raw fields of the alert. Based on expert experience, he performs queries on operational data (such as network traffic, and host logs) and public threat intelligence databases (such as the black IP, the black domain name, and the Indicators of Compromise (IoC)). Through repeated comparisons and multi-party verification then he makes a decision on whether there is a cyberattack at the moment.

For analysts, the three key actions of the human-in-the-loop alert validation process are: (1) making a decision, (2) assigning labels, and

(3) assessing severity and priority. Analysts can leverage all available information to determine whether the alert is an attack or non-attack alert, including threat score, alert type, IP addresses, attack timeline, security domain information, background log, etc. In addition, analysts also add some labels, such as *false positives*, *keep tracking*, *failed attack attempt*. Especially for multi-step attacks, the attack time span is long and the attack methods are complex. When related alerts occur in the future, these labels act as memos to remind analysts. Also, analysts would correct the possible errors that are generated by systems or models automatically, such as changing the *attack_stage* field of raw alerts. Finally, analysts assess the severity and priority intuitively.

In summary, expert experience is crucial in the process of the alert validation process. Analysts rely on their knowledge and expertise in cybersecurity to make informed decisions about whether an alert signifies an attack. This experience helps them understand the nuances and patterns of different types of threats, enabling them to accurately assess the severity and response priority of an alert.

In this paper, by incorporating expert knowledge and experience, AlertPro extracts 8 history features from expert feedback and employs an active learning model to effectively filter out low-risk alerts and highlight high-risk ones. These 8 history features are listed in Table 2 and explained in Section 4.2.3.

2.5. Problem statement and research questions

For alert set $A = (a_1, a_2, \dots, a_n)$ — a_i is the i th alert. The set of attack alerts is A_a . Specifically, as in Equation (1), the anomaly scoring model M calculates the score S for each alert in set A and delivers the alert a_i with the highest score s_i to the analyst. Assuming the model M is reliable, this selected alert a_i should belong to the attack alert set A_a . Our goal is to deliver a maximum number of attack alerts to the analyst within the budget B times of feedback interactions.

$$\text{maximum} \left(\sum_{i=1}^B (a_i \in A_a \text{ and } s_i = \max(S) : S = M(A)) \right) \quad (1)$$

Based on the problem identified, we have crafted the following three research questions that will guide the design (Section 4), implementation and evaluation (Section 5) of AlertPro.

RQ1: Can context features enhance the accuracy of multi-step attack detection in anomaly detection?

RQ2: Can active learning models incorporating history features improve the attack discovery rate compared with state-of-the-art anomaly detection algorithms?

RQ3: Is AlertPro a practical solution considering the performance and overhead?

3. Related work

We present related work from two aspects. The first aspect is to discuss alert analysis techniques from the perspective of the typical SOC workflow. The second aspect is to introduce the active anomaly discovery methodology with expert feedback from the perspective of machine learning models.

3.1. Alert analysis techniques

In Table 1, we summarize the work related to alert analysis techniques based on different dimensions, such as the SOC stage where it is applied, the machine-learning methods used, features utilized in machine-learning models, whether it is designed for multi-step attack detection, whether the model training process incorporates expert efforts, and whether it can be transferred to new situations without the need for retraining.

The existing alert analysis techniques are as follows. (1) Traditional classification algorithms, either the supervised Weighted Support Vector Machine (WSVM) or the unsupervised Isolation Forest (IF) are used

Table 1

Comparison of related work. AlertPro has 3 main advantages: detecting multi-step attacks, incorporating expert feedback, and no need to retrain.

Research	SOC_stage	Method	Feature	Detecting multi-step attack	Incorporates expert effort	No need to retrain
Ban et al. (2021)	Prioritization	WSVM	Basic	×	×	×
Aminanto et al. (2019)	Prioritization	Isolation forest	Basic	×	×	×
Siddiqui et al. (2019)	Triage, Validation	Isolation forest	Custom*	×	✓	✓
DeepCASE (van Ede et al., 2022)	Prioritization, Validation	Attention mechanism	N/A**	×	✓	×
Context2Vector (Liu et al., 2022a)	Prioritization, Validation	LDA2Vec	Custom	×	✓	×
AI2 (Veeramachaneni et al., 2016)	Prioritization, Validation	Ensemble learning	Not List	×	✓	×
Ning et al. (2002); Shittu et al. (2014); Husák and Kašpar (2019)	Correlation	Rule-based graph	N/A	✓	×	×
SGAE (Nadeem et al., 2021)	Correlation	Connection graph	N/A	✓	×	×
GAC (Haas and Fischer, 2018)	Correlation	Connection graph	N/A	✓	×	×
Hassan et al. (2019); Liu et al. (2022b)	Correlation	Provenance graph	N/A	✓	×	×
AlertPro	Prioritization, Validation	Isolation forest, Active learning	Basic, Context, History	✓	✓	✓

* Custom refers to using self-defined features.

** N/A refers to not applicable.

to sort alerts (Aminanto et al., 2019; Ban et al., 2021; Siddiqui et al., 2019). (2) Deep learning models (such as the attention mechanism, LDA2Vec), are also used to integrate contextual information between alerts and events in DeepCASE (van Ede et al., 2022) and Context2Vector (Liu et al., 2022a). DeepCASE is a semi-supervised approach that aggregates events into multiple classes and delivers them to security experts for analysis after class sampling. Context2Vector (Liu et al., 2022a) uses the LDA2Vec model to generate topic representations of alert sequences for classification. LDA2Vec is a machine learning algorithm that combines two popular techniques: Latent Dirichlet Allocation (LDA) and Word2Vec. AI2 (Veeramachaneni et al., 2016) combines the results of unsupervised and supervised models to identify attack events, then presents anomalous events to the analyst for investigation. (3) Alert correlation primarily relies on graph processing techniques. The graph generation algorithm has 3 branches: rules-based graph (Ning et al., 2002; Shittu et al., 2014; Husák and Kašpar, 2019), connection graph (Nadeem et al., 2021; Haas and Fischer, 2018), provenance graph (Hassan et al., 2019; Liu et al., 2022b). The rule-based graph is generated through predefined correlation rules (Ning et al., 2002; Shittu et al., 2014; Husák and Kašpar, 2019). SAGE (Nadeem et al., 2021) generates alert attack graphs from alerts without prior knowledge through suffix-based probabilistic deterministic finite automaton (S-PDFA). GAC (Haas and Fischer, 2018) combines alerts with alert-related data such as system logs or IoC data to provide a more in-depth analysis. Using process-level system logs as supplementary data, approaches (Hassan et al., 2019; Liu et al., 2022b) correlate alerts to generate dependency graphs, and finally assign risks for each alert.

However, from the effective alert prioritization and validation in multi-step attack scenarios, there are a few drawbacks. (1) In traditional classification algorithms, the thresholds need to be adjusted to find the optimal one (Zhou and Liu, 2005; Yu et al., 2015; Lyu et al., 2022). Moreover, methods applied in (Ban et al., 2021; Aminanto et al., 2019) that only use basic features may result in a high false positive rate due to the lack of consideration for attack context. (2) Though DeepCASE (van Ede et al., 2022) and Context2Vector (Liu et al., 2022a) contain humans to label analysis results, analysts only serve to annotate the alert type, experts still need to select from alert clusters for labeling, and the model does not use human feedback to improve further results. AI2 (Veeramachaneni et al., 2016) needs to constantly retrain the supervised model using the analyzed alerts. (3) Discovering new patterns and updating correlation rules can be challenging (van Ede et al., 2022; Liu et al., 2022b), and generating a process-level graph is time-consuming (van Ede et al., 2022). Alert correlation analysis is more suitable for cyberattack investigation (Navarro et al., 2018b).

Compared with existing work, AlertPro has 3 main advantages: (1) AlertPro is designed for multi-step attack detection, which benefits from

the context features extracted from alert sequences. (2) AlertPro incorporates expert feedback and employs an active learning model to effectively filter out low-risk alerts and highlight high-risk ones, which benefits from the history feature extracted from those alerts that have been investigated by analysts. (3) AlertPro can be transferred to new situations without the need for retraining, which is evaluated in Section 5.6.

3.2. Active anomaly discovery with expert feedback

The related research of active anomaly discovery with expert feedback is as follows. (1) Online learning updates the model after every feedback interaction. An approach proposed in Siddiqui et al. (2018a) formulates feedback-guided anomaly detection as an online convex optimization (OCO) problem by adjusting the weight vector from feedback responses. (2) Active learning models can adaptively incorporate professional perspectives through expert feedback, improving the accuracy of the model over time. Models proposed in Das et al. (2016); Zha et al. (2020); Siddiqui et al. (2018b); Lamba and Akoglu (2019) find anomalies related to the analyst's interest by querying the analyst's feedback. AAD (Das et al., 2016) employs the Accuracy at the Top (AATP) approach to update the model, while Meta-AAD (Zha et al., 2020) uses the Proximal Policy Optimization 2 (PPO2) algorithm to improve anomaly detection.

In this paper, we try to combat the alert fatigue problem. The crucial solution is prioritizing alerts based on their potential severity, enabling analysts to allocate their efforts accordingly. The goal of active anomaly discovery is to present to the expert the maximum number of anomaly samples in constant interaction, which is consistent with the problem statement in Section 2.5. Inspired by AAD (Das et al., 2016) and Meta-AAD (Zha et al., 2020), AlertPro utilizes anomaly detection to initially rank alerts, and subsequently employs active learning, incorporating expert feedback to refine the alert ranking. The overview of AlertPro is shown in Fig. 5 and explained in Section 4.1.

4. Design of AlertPro framework

AlertPro is designed to alleviate the burden on security analysts by automatically prioritizing alerts and identifying high-risk alerts that require further manual examination. The system incorporates the human-in-the-loop approach, allowing it to gain valuable insights from expert feedback and thereby improve its accuracy in detecting critical alerts. This section first introduces the design overview in Section 4.1 and then presents its three key modules: feature extraction, rank by anomaly detection, and re-rank by active learning in Section 4.2, 4.3, and 4.4, respectively.

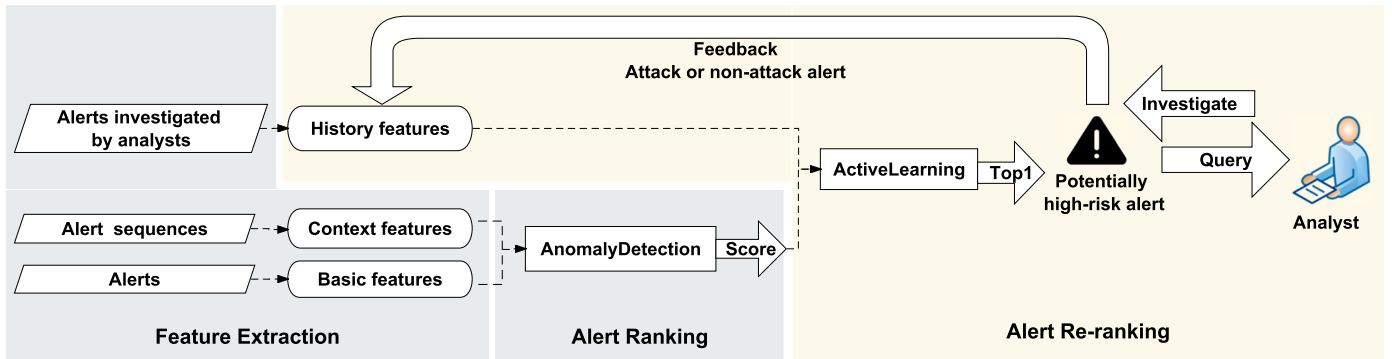


Fig. 5. Overview of AlertPro. The closed-loop alert analysis process is feature extraction, alert ranking, and alert re-ranking. In the feature extraction module, three types of features are extracted from different alerts. The alert ranking module provides anomaly scores using basic features and context features. In alert re-ranking, the module re-ranks alerts through active learning to find potentially high-risk alerts and update history features by the analyst feedback.

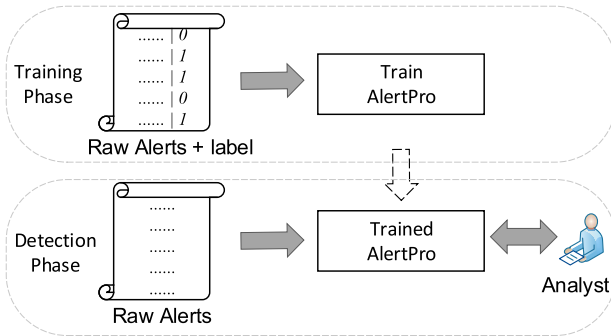


Fig. 6. AlertPro Development Phases. In the training phase, the model is trained using labeled datasets. In the detection phase, the trained model interacts with analysts to rank raw alerts according to their potential risk.

4.1. Overview

The overview of AlertPro is shown in Fig. 5 which contains three main modules. **Feature extraction** module extracts basic features from raw alerts, context features from alert sequences, and history features from alerts that have been investigated by analysts. These features are detailed in Section 4.2. **Alert Ranking** module utilizes basic features and context features as input, employs the IF algorithm to calculate the anomaly score for each alert, and initially ranks alerts based on the anomaly detection score. This module is detailed in Section 4.3. **Alert Re-ranking** module uses history features and anomaly detection scores as input, employs reinforcement learning algorithms, and incorporates expert feedback to re-rank alerts. The purpose is to prioritize the highest-risk alerts for analysts whenever they make inquiries.

Fig. 6 illustrates the development process of AlertPro, which contains two phases: the training phase and the detection phase. During the training phase, the model is trained using datasets with associated labels. In the detection phase, the trained model is applied to rank raw alerts based on their potential risk. The highest-ranked alert is presented to experts and the feedback is used to re-rank alerts.

4.2. Feature extraction

Table 2 provides a description of the features used by AlertPro, along with whether they were used in previous studies. It is worth noting that the 7 context features are proposed by us for the first time. Previous studies primarily focused on basic features to rank or classify alerts and neglected the inter-correlation between alerts. However, in multi-step attack detection, the inter-correlation between alerts is crucial. Therefore, AlertPro takes advantage of both basic and context features to calculate anomaly scores. Further, AlertPro also incorporates historical

features in its active learning model, achieving a context-aware attack detection model with enhanced performance.

4.2.1. Basic feature explanation

Basic features are several fields that are selected from raw fields of raw alerts (as shown in Fig. 2), which can differentiate this alert from others. There are 4 basic features listed in the 1 – 4 row of Table 2.

Feature 1 and 2: Alert subject - Source IP (SIP) and Destination IP (DIP). A subject represents a participant in the alert, either an attacker or a victim. In this paper, alerts are generated by network activities so that the subject represents the pair of (*source IP*, *destination IP*). For alerts triggered by the process behavior and file operation on the host, the subject represents the pair of (*process path*, *process name*), (*file path*, *file name*) respectively, which can also be handled by AlertPro. We use frequency encoding to replace the original alert subject, which is the statistical characteristic of each subject. In contrast to the one-hot encoding method employed in previous research (Ban et al., 2021; Aminanto et al., 2019), frequency encoding can effectively retain the frequency relationships of IP occurrences without causing sparsity issues.

Feature 3: Alert signature. The signature describes the specific method by which the attack is executed, allowing for a detailed classification of the alert. Alert signatures are typically presented as human-readable text, so vectorization is necessary to utilize them as features in machine learning models. Likewise, we employ Frequency Encoding to vectorize the alert signature feature.

Feature 4: Attack stage. The attack stage of an alert provides information about the progression of the attack in the multi-step attack process. According to the Cyber Kill Chain (Hutchins et al., 2011), we divide the multi-step attack process into five stages: reconnaissance, exploitation, get access privilege, C&C, and post-penetration. Due to the limited number of attack stages, it is suitable to use a one-hot encoding.

4.2.2. Context feature explanation

Context features are extracted from a sequence of alerts triggered by one multi-step attack. To clarify, an alert sequence refers to a collection of alerts that share the same alert subject within a specific time interval and are arranged in chronological order. While a single alert may initially appear low-risk when evaluated individually, within the context of a multi-step attack, it may escalate to a critical level, requiring a more thorough analysis. To enhance the detection rate of multi-step attacks, we extract contextual features to capture these contextual relationships. Drawing upon expert experience in real-world alert analysis, we select 7 context features listed in the 5 – 11 row in Table 2.

Feature 5: # Alerts generated by the same subject in all alerts. This feature measures the number of alerts that share the same attacker or victim with this alert in all alerts. If this number is large, it implies two situations: either these alerts are false positives of normal behavior, or attackers are concentrating their fire on one target.

Table 2

Summary of features. There are 3 categories and 19 features in total. Basic features are extracted from fields of raw alerts. Context features are extracted from alert sequences. History features are extracted from alerts that have been investigated by analysts.

Seq	Type	Feature descriptions	Usage	Previous adoption
1	Basic	Alert subject - Source IP (SIP)	Calculating anomaly score	Husák and Kašpar (2019); Ban et al. (2021); Shittu et al. (2014); Aminanto et al. (2019); Nadeem et al. (2021)
2	Basic	Alert subject - Destination IP (DIP)	Calculating anomaly score	Husák and Kašpar (2019); Ban et al. (2021); Shittu et al. (2014); Aminanto et al. (2019); Nadeem et al. (2021)
3	Basic	Alert signature	Calculating anomaly score	Husák and Kašpar (2019); Ban et al. (2021); Shittu et al. (2014); Aminanto et al. (2019); Nadeem et al. (2021)
4	Basic	Attack stages	Calculating anomaly score	Husák and Kašpar (2019); Ban et al. (2021); Aminanto et al. (2019); Nadeem et al. (2021)
5	Context	# Alerts generated by the same subject in all alerts	Calculating anomaly score	None
6	Context	# Identical alerts in all alerts	Calculating anomaly score	None
7	Context	# Alerts in the sequence	Calculating anomaly score	None
8	Context	# Identical alerts in the sequence	Calculating anomaly score	None
9	Context	# Attack stages in the sequence	Calculating anomaly score	None
10	Context	# Alert of each attack stage in the sequence	Calculating anomaly score	None
11	Context	# Attack stages on the victim in the sequence	Calculating anomaly score	None
12	History	Whether the signature in attack alerts	Re-ranking based on feedback	None
13	History	Whether the signature in non-attack alerts	Re-ranking based on feedback	None
14	History	Whether the SIP or DIP in attack alerts	Re-ranking based on feedback	None
15	History	Whether the SIP or DIP in non-attack alerts	Re-ranking based on feedback	None
16	History	Whether there are attack alerts in top N distance	Re-ranking based on feedback	Zha et al. (2020)
17	History	Whether there are non-attack alerts in top N distance	Re-ranking based on feedback	Zha et al. (2020)
18	History	Mean distance with attack alerts	Re-ranking based on feedback	Zha et al. (2020)
19	History	Mean distance with non-attack alerts	Re-ranking based on feedback	Zha et al. (2020)

Feature 6: # Identical alerts in all alerts. This feature measures the count of identical alerts that match the current alert in all alerts. Identical alerts are those that share the same alert subject and signature. The cognitive tendency is that an increased count of identical alerts is indicative of a lower-risk alert. Frequent occurrences of redundant alerts could suggest repetitive triggering events. Typically, such alerts are often considered low severity as they lack unique characteristics. For example, policy violations such as *weak password login* alerts may be normal daily login behavior.

Feature 7: # Alerts in the sequence. This feature measures the number of alerts that belong to the same sequence as this alert. An alert sequence refers to a collection of alerts that share the same subject over a time interval. The relationship between the number of alerts in a sequence and the sequence significance is not straightforward. A sequence that includes a large number of alerts belonging to the same attack stage is deemed less significant compared to a sequence that may have a smaller number of alerts but covers multiple attack stages. Hence, we propose features 8–11 to complement the features about the attack stage, as they play a crucial role in multi-step attacks.

Feature 8: # Identical alerts in the sequence. This feature measures the count of identical alerts with the current alert in the same alert sequence. It follows that, within such a sequence, the severity of alerts decreases as their occurrence frequency increases.

Feature 9: # Attack stages in the sequence. This feature quantifies the count of attack stages within the current alert sequence. Analytically, the presence of multiple attack stages within a sequence can indicate a relationship between the alerts in that sequence and actual attacks. An increasing count of distinct attack stages within a sequence raises the probability that it represents a complete attack.

Feature 10: # Alerts of each attack stage in the sequence. This feature measures the number of alerts in each attack stage within the current alert sequence. The number of alerts in each attack stage in the sequence is also related to the severity of the alert. A higher count of alerts in later attack stages suggests a heightened likelihood of a critical attack. For example, the alerts focused on the reconnaissance stage are not as severe as those focused on the C&C stage.

Feature 11: # Attack stages on the victim in the sequence. This feature measures the number of attack stages on the victim within the current alert sequence. Here, we make the assumption that the destination IP represents the victim. The identification of multiple alert stages targeting the victim enhances the likelihood of confirming a critical attack.

4.2.3. History feature explanation

History features are extracted from alerts that have been investigated by analysts. These features are utilized as the state in reinforcement learning, with more details in Section 4.4. As the alert validation process explained in Section 2.4, expert experience is crucial in the SOC workflow. Incorporating expert feedback can effectively filter out low-risk alerts, especially false positives subsequently. For example, if an alert receives a high score in anomaly detection and shares similar features with another alert that was previously classified as a false alert by analysts, it is also likely that the new alert is a false alert. These 8 history features are listed in the 12 – 19 row in Table 2.

Feature 12 and 13: Whether the signature is in attack alerts or non-attack alerts. These two features measure whether alerts with the same signature have been verified as attack alerts or non-attack alerts by analysts, respectively. Alerts with the same signature are generally triggered by the same detection rule. If the alert generated by this rule is verified as an attack alert, it means that this rule is important and accurate. Furthermore, if an alert with a specific signature has been previously verified as an attack alert by analysts, it can serve as evidence that future alerts with the same signature are also likely to be attack alerts. Hence, if the alert with the same signature has been confirmed as an attack alert, AlertPro assigns Feature 12 with the numerical value 1. Similarly, if the alert with the same signature has been confirmed as a non-attack alert, AlertPro assigns Feature 13 with the numerical value 1. And if the alert with the same signature has not been analyzed, AlertPro assigns both features with the numerical value 0.

Feature 14 and 15: Whether the SIP or DIP in attack alerts or non-attack alerts. These two features measure whether the alerts with the same subject, either SIP or DIP, have been verified as attack alerts or non-attack alerts, respectively. Alert with the same subject indicates that the alert is triggered by the actions of the same entity. If a previous alert triggered by the same subject is verified as an attack alert by analysts, it indicates that the corresponding entity is sensitive to attacks. Feature 14 and 15 are assigned numerical values of 1 and 0, respectively, just like features 12 and 13.

Feature 16 and 17: Whether there are attack or non-attack alerts in the top N distance. These two features measure whether there are attack or non-attack alerts verified by analysts in the top N distance nearest alerts (Zha et al., 2020), respectively. If there is an attack alert nearby, there is a high possibility that this alert is an attack alert. The distance between alerts is measured by calculating the Euclidean distance between alert vectors. Feature 16 and 17 are assigned numerical

values of 1 and 0, respectively, just like features 12 and 13. In Section 5.5, we set $N = 5$ in the evaluation experiment empirically.

Feature 18 and 19: Mean distance with attack or non-attack alerts. These two features measure the mean Euclidean between this alert and all attack alerts or non-attack alerts verified by analysts (Zha et al., 2020), respectively. The intuition behind these features is that if an alert is located far away from attack alerts, it is less likely to be an attack alert. Feature 18 and 19 are assigned numerical values between 0 to 1, respectively.

4.3. Alert ranking using anomaly detection

Similar to anomaly detection, attack alerts constitute only a small portion of all alerts and exhibit distinct characteristics compared to false positives or irrelevant alerts. Therefore, ranking alerts can be viewed as anomaly detection tasks, where attack alerts are treated as anomalies, and all other alerts are considered normal data. Initially, AlertPro utilizes an unsupervised anomaly detection algorithm, such as the isolation forest or one-class SVM, as the foundational model to calculate anomaly scores for each alert in the alert pool.

4.4. Re-ranking using active learning

Once the anomaly scores are calculated, combined with history features, AlertPro re-ranks the alerts using active learning models. Active learning is a supervised machine learning approach that involves an iterative process of selecting and labeling the most informative instances from a large pool of unlabeled data. Active learning query policy refers to the strategy or rule used to select the most informative instances for labeling during the active learning process. The query policy determines which unlabeled instances should be presented to human annotators or domain experts for labeling, with the goal of maximizing the learning efficiency and model performance. In this paper, AlertPro adopts a query policy that exclusively outputs attack alert candidates for validation by analysts. The results obtained from this validation process are then utilized back to adjust the model's subsequent outputs. We implement active learning feedback using reinforcement learning inspired by Meta-AAD (Zha et al., 2020) which aims to get maximum long-term benefits from the anomaly discovery process. We have re-designed the state and reward functions to further perform the task of alert re-ranking. The properties of reinforcement learning in the alert re-ranking are described as follows.

State includes the anomaly score and history features. The anomaly score is the score calculated by the anomaly detection algorithm, and the value range is $[0, 1]$. The history features of the alert include whether an alert with the same signature or subject is previously verified as an attack or non-attack alert, whether there are surrounding attack alerts and non-attack alerts, and the average distance to the verified attack and non-attack alerts respectively. In total, there are 9 dimensions to describe the state of an alert: anomaly score and 8 history features.

Action is a discrete value, 1 or 0, that indicates whether or not to present this alert to analysts in one query. The alert presented to analysts is the one that is predicted as a high-risk attack by the active learning model.

Reward is adjusted based on its performance in predicting whether an alert is an attack or not. The reward value for the active learning actions depends on the feedback and confirmation from analysts. These rewards are designed to inform predictive actions and provide additional guidance for future predictions. The calculation method for determining rewards is presented in Equation (2).

$$\text{reward} = \begin{cases} 0, & \text{ignored non-attack alerts} \\ \text{score}, & \text{TP and in top 10\%} \\ 1, & \text{TP and out top 10\%} \\ -0.1 \times \text{score}, & \text{FP and between top and bottom 10\%} \\ -0.1, & \text{FP and in bottom 10\%} \end{cases} \quad (2)$$

TP: True Positive; FP: False Positive

The top $N\%$ of alerts refer to the alerts whose anomaly score fall within the highest $N\%$. In the paper, we set $N = 10$ empirically. In each query, the alerts that the model predicts non-attacks would not present to analysts and thus will be rewarded with 0. If the model predicts that the alert is an attack, and analysts verify the prediction is correct, i.e. true positive, the model is rewarded with a positive reward. For this alert, if it scores in the top 10%, the reward is equal to the anomaly score which is in $[0, 1]$. Or if it scores out of the top 10%, the model is still rewarded with a positive reward of 1, which encourages the model to explore alerts with lower anomaly scores. For false positives, i.e., alerts that the model wrongly predicts as an attack, the model is penalized with a negative reward. If the anomaly score is between the top and bottom 10% of all scores, the model is penalized with a negative reward equal to $-0.1 \times \text{score}$. For anomaly scores within the bottom 10%, the model is penalized with a fixed negative reward of -0.1 , which discourages it from exploring alerts with the lowest anomaly scores. Overall, the reward system is designed to strike a balance between encouraging the model to focus on the alerts with anomaly scores in the mid-range and discouraging exploration of alerts with the highest and lowest scores.

The AlertPro alert feedback process is shown in the Algorithm 1. The algorithm's input is raw alerts and *Budget*. The output is the top suspicious attack alert in *Budget* interactions. Before the training phase, the feature extraction function *GetFeatures* extracts the alert features, and then the anomaly detection function *AnomalyScore* calculates the anomaly score for each alert. During the training phase, the model M selects an action through the policy based on the state and then updates the policy according to the received reward. In the detection phase, based on the current state and trained policy, the model M selects an alert with a high likelihood of being an attack alert. It also updates the history features based on feedback from experts. This process involves recording instances, signatures, and subject information. In the next round, real-time history features are extracted as the current environment state. The model M can be used for both individual detection and batch detection, depending on the number of alerts returned each time.

Algorithm 1 Alert Feedback Process.

Input: *RawAlerts*, *Budgets*
Output: *SuspiciousAttackAlerts*

```

1:  $Feature_a \leftarrow GetFeatures(RawAlerts)$ ;
2:  $Score_a \leftarrow AnomalyScore(Feature_a)$ ;
3: for  $ts < Timesteps$  do ▷ training phase
4:    $State \leftarrow Score_a + HistoryFeatures$ 
5:   Choose Action based on Policy
6:   Get Reward by Equation (2)
7:   Save Feedback Labels
8:   Update HistoryFeatures
9:   Update Policy
10: end for
11: for  $iteration < Budgets$  do ▷ detection phase
12:    $State \leftarrow Score_a + HistoryFeatures$ 
13:   Compute Priority for Each Alert
14:   Save Feedback Labels
15:   Update HistoryFeatures
16: end for

```

The simplicity of the environment's state and discrete action space makes this approach well-suited for various reinforcement learning models, such as Dueling Double Deep Q Network (D3QN) (Van Hasselt et al., 2016; Wang et al., 2016), Advantage Actor-Critic (A2C) (Mnih et al., 2016), and Proximal Policy Optimization (PPO2) (Schulman et al., 2017). D3QN is an extension of DQN, which is the most popular value-based algorithm. D3QN solves the problem of overestimating action values and speeds up the training process by combining Double DQN and Dueling DQN. A2C and PPO2 are policy gradient methods that can handle both continuous and discrete action spaces. Performance evaluation of these candidate models is presented in Section 5.2.

Table 3

Summary of original datasets. These datasets were chosen or prepared to meet three requirements: (1) containing alerts or raw data capable of generating alerts, (2) encompassing multi-step attacks, and (3) providing ground truth for the attacks involved.

Dataset	Raw size [*]	Background noise ^{**}	# All alerts	# Attack alerts	Ratio	# Attack IP	# Victim IP	# Total IP
Ildos	116 MB	-	750	51	6.80%	1	3	95
iscx	3.95 GB	-	12463	107	0.86%	4	4	538
CPTC-T1	39710 PCS	2021.1.1	85548	1609	1.88%	27	29	61794
CPTC-T5	18447 PCS	2022.1.1	142463	2098	1.47%	25	26	95707
CPTC-T8	15560 PCS	2023.1.1	73046	2067	2.83%	28	29	55836

^{*} MB and GB are the size of raw packets, and PCS means the pieces of raw alerts.

^{**} We use the collection dates to identify daily traces from WIDE MAWI traffic datasets.

5. Implementation and performance evaluation

In this section, we begin by introducing the practical implementation of AlertPro which includes detailed information on data preparation in Section 5.1, as well as the experiment setup and the choice of evaluation metrics in Section 5.2. Subsequently, we present a detailed case study demonstrating how AlertPro effectively uncovers undisclosed attacks in Section 5.3. Following this case study, in response to 3 research questions proposed in Section 2.5, we evaluate AlertPro's performance in Section 5.4, 5.5, and 5.6.

5.1. Data preparation

Datasets that fulfill the design principles of AlertPro should satisfy three requirements: (1) containing alerts or raw data capable of generating alerts, (2) encompassing multi-step attacks, and (3) providing ground truth for the attacks involved. Firstly, given the focus of this study on alerts, it is crucial to ensure that the dataset includes either alerts or raw data that is capable of generating alerts. One alternative approach is to generate alerts using security tools based on the raw data, such as network packet captures (PCAP). NIDS tools like *snort*³ and *Suricata*,⁴ and EDR software like *Symantec EDR* can be used to generate alerts. Secondly, the dataset should include multi-step attacks to ensure that the alerts exhibit a logical and comprehensive attack strategy that analysts can follow, rather than isolated single attacks. Finally, in order to effectively evaluate the performance of AlertPro, the dataset should provide a detailed description of the attack process. This ground truth also serves as valuable feedback for analysts.

Since public datasets for alert-based detection are not available, it is necessary for us to modify existing datasets in order to construct alert datasets. The statistics of these five alert datasets are presented in Table 3. The datasets used in the experiments are described as follows. (1) DARPA 2000 LLDOS1.0 dataset (*Ildos*) (MIT Lincoln Lab, 2000). Despite being outdated and relatively small, the *Ildos* dataset is still widely used due to being fully labeled and publicly available. The *Ildos* dataset is PCAP data, and we use *Snort 2.9* with *Emerging Threats* rules to generate alerts. (2) UNB ISCX IDS 2012 dataset (*iscx*) (Shiravi et al., 2012). Similar to the *Ildos* dataset, the *iscx* dataset is also PCAP data and we deploy *Snort 2.9* with *Emerging Threats* rules to generate alerts. (3) Collegiate Penetration Testing Competition (CPTC) 2018 dataset (Munaiah et al., 2019). The CPTC dataset contains alerts generated by *Suricata*. CPTC is a red-team penetration test in which multiple teams attempted to hack an automotive company.

Specifically, these datasets employed in the experiments encompass various types of attacks and multi-step attack sequences, which will be described in detail below.

Ildos. The attacker first scans the network segment to identify surviving hosts and then performs *Sadmind Ping* on each of the identified

hosts to locate the host running the *Sadmind* service. After identifying the host, the attacker exploits the *Sadmind* Buffer Overflow vulnerability on 3 valid hosts to gain root privileges. Finally, these 3 hosts are used to launch Distributed Denial of Service (DDoS) attacks on the target.

iscx. We selected the attack scenario of infiltrating the network. The attacker first sends a phishing e-mail to the victim with a malicious PDF file as an attachment. This file exploits a buffer overflow vulnerability in Adobe Reader. The attacker then sets up the *Meterpreter* Reverse TCP shell to listen on port 5555. After the victim host *USER5* reverse connects to the attacker, the attacker uses it to scan two consecutive subnets to search for surviving hosts and exploitable services. After discovering the alive host *USER12* with the vulnerable SMB authentication protocol on port 445, the attacker exploits this vulnerability to gain *USER12* privileges and scans its subnet from *USER12*. The attacker discovers the target server running MS SQL Server as the back-end database and then exploits SQL injection through *USER12*, creates a new user, and deletes the original user, finally gains control of the target server.

CPTC. We selected three attacker teams, namely *T1*, *T5*, and *T8*, who successfully achieved data exfiltration. These teams exploited the vulnerabilities of the target host in different ways, involving service and vulnerability discovery, privilege escalation, remote code execution, and data theft. Since most alerts in this dataset are generated by penetration testing attacks, we used traffic from the WIDE MAWI backbone network project⁵ as background data to enrich this dataset. Similarly, we used *snort 2.9* to generate noise alerts in the background traffic, i.e., false positives and irrelevant alerts. This emulates a more realistic real-world scenario. Using traffic from a real-world network as background data is a common practice in intrusion detection research as it can provide a more realistic and diverse set of traffic patterns (Fu et al., 2021; Nisioti et al., 2018).

Merging alerts using pre-defined rules to reduce analysts' workload is the first step in analysis work. We merge the alerts that have the same source IP, destination IP, and signature_id, and continuously appear in a 10-minute time window. After that, according to the ground truth, we manually labeled the alerts with either 0 (*non-attack*) or 1 (*attack*) as security analysts' feedback during interactions.

Furthermore, considering the information provided in Table 3, it can be inferred that the alert ranking aligns with the goals of an anomaly detection task. Anomaly detection techniques are frequently utilized to identify patterns of behavior that deviate from the expected norms in a monitored network, with the assumption that such deviations indicate genuine attack alerts. Hence, by utilizing anomaly detection methods, it becomes feasible to detect abnormal behavior patterns that may indicate an ongoing attack.

5.2. Experiment setup and evaluation metrics

AlertPro is implemented using *Python 3*, with the IF algorithm provided by the *sklearn* library and the reinforcement learning model by

³ <https://www.snort.org/>.

⁴ <https://suricata.io/>.

⁵ <https://mawi.wide.ad.jp/mawi/>.

*Stable-Baselines*⁶. The experiments are performed on a Ubuntu server with Intel Xeon E5-2683 2.1GHz CPU (64 processors) and 768GB memory physically, of which approximately 85GB memory is actually used by AlertPro.

We evaluated the performance of our anomaly detection approach using three widely adopted metrics (Powers, 2020): Precision (P), Recall (R), and F1-score (F1). Considering the class imbalance in the datasets, Precision and Recall are the most important metrics which can measure and represent the ability to correctly detect positive instances, i.e., attack alerts.

For evaluating the performance of interactive active learning, the Anomaly Discovery Curve is a commonly used metric. This curve measures the proportion of real anomaly instances in all instances presented to experts. The horizontal axis of the Anomaly Discovery Curve is the number of interactions, and the vertical axis is the number of attack alerts discovered during the interaction. To quantify the Anomaly Discovery Curve, we use two evaluation metrics proposed in Ding et al. (2019): Cumulative Precision (cP) and Cumulative Recall (cR). In Equation (3), cP measures the percentage of discovered real attack alerts found in *Budget* interactions. Similarly, in Equation (4), cR measures the percentage of discovered real attack alerts out of all alerts. These metrics collectively provide valuable measures for assessing the performance of interactive active learning.

$$cP = \frac{\text{Discovered Attack Alerts}}{\text{Budget}} \quad (3)$$

$$cR = \frac{\text{Discovered Attack Alerts}}{\text{All Attack Alerts}} \quad (4)$$

To evaluate AlertPro, we first offer a detailed explanation of how AlertPro effectively discovered the undisclosed attack.

5.3. Case study of how AlertPro can discover an undisclosed attack in *iscx* dataset

We employ AlertPro's alert analysis on the *iscx* dataset as an illustrative example to introduce the evaluation process. **Anomaly Detection Ranking.** Following feature extraction, AlertPro uses the IF algorithm to rank alerts. Among the top *Budget* (110) alerts, 71 alerts belong to actual attacks. Within these alerts, AlertPro successfully detects an unknown attack. This attack involved the exploitation of various vulnerabilities in an attempt to gain root privileges. The specific exploit attempts included an FTP username or password overflow attack, *fp30reg.dll* file buffer overflow attack on Microsoft FrontPage 2000 server,⁷ etc.

Feedback Re-ranking. After initial anomaly detection, AlertPro begins expert iterative feedback. Although initial anomaly detection discovered an unknown attack, the attack mentioned in ground truth ranks lower and is more dispersed, with the highest ranking at 97th and the lowest ranking at 1871st. During the interaction, AlertPro first discovers comprise alerts between *USER12* and the target server, originally ranked at 97th. Then AlertPro discovers the abnormal communication between *USER5* and *USER12*. Finally, AlertPro identifies two key alerts in the attack progression: *USER5* and *USER12* connect back to the remote attack IP (ranking 1438th and 1622nd in the original anomaly detection). Throughout the attack, *USER5* and *USER12* only connect to the attackers once, and later these two hosts are controlled to execute the subsequent attack. Thus there are only two alerts related to the actual attack IP, both of which are discovered by AlertPro. This analysis process aligns with the principles of analyst analysis: originating from confirmed high-risk alerts, and discovering the attack related to these alerts. For non-attack alerts with high scores, including *ssh scan*, *dns query to a suspicious domain*, AlertPro can exclude these non-attack

alerts through interaction with expert feedback. Among 110 interactions, AlertPro only raises 7 false positive non-attack alerts. Some of these are linked to compromised hosts, such as *sensitive DNS query* performed by the compromised host after being exploited. Although false positives, these alerts still need attention, indicating potential follow-up actions from the attacker. AlertPro can send these alerts to experts for further validation. Once validated, such alerts are no longer at high-risk.

From the above process, AlertPro is capable of assigning a high rank to alerts related to this attack in the anomaly detection phase. Subsequently, during the active learning re-ranking stage, AlertPro sustains the high ranking of these alerts, gradually identifies real attack alerts, and reduces the ranking of irrelevant alerts. Consequently, upon query, these alerts are presented to analysts for thorough investigation. Significantly, all the decisions made in the above steps are correct, leading to the detection of this hidden attack. It is noteworthy that AlertPro successfully identified all alerts associated with this undisclosed attack.

5.4. Feature effectiveness (RQ1)

To answer RQ1, we assess the efficacy of various features in detecting alert anomalies. We choose the IF algorithm (Liu et al., 2008), which represents state-of-the-art anomaly detection. Through an ablation study, the module evaluates the relative importance of different types of features by examining their performance in anomaly detection. Specifically, we evaluate the performance of basic and context features in this module, while history features are assessed in the active learning module in Section 5.5. The contribution of each type of feature to attack detection is presented in Table 4, with each experiment result being an average of 10 runs. We find the optimal threshold through the Receiver Operating Characteristic (ROC) curve. When the recall rate approaches 1, a low precision rate indicates that many false positive alerts are being classified as attack alerts.

Based on Table 4, we can conclude that employing context features yields better results than using only basic features, particularly in precision and F1-score. Although the recall of context features in *lldos* is lower compared to using only basic features, the results of using context features in the other two indicators have significantly improved, with a precision increase of 38.83% and the f1-score increase of 21.82%. In the case of the *lldos* dataset, high recall (> 90%) and low precision (< 50%) of basic features further signify that relying solely on basic features leads to misidentifying scanning and exploit attempt alerts without subsequent actions as high-risk alerts. The performance of using all features and context features is generally comparable, and the F1-score of all features in the CPTC dataset exceeds the context features. In the case where the context features are not effective in detecting the attack, basic features can enhance the overall detection outcome. Furthermore, the basic features can help in situations where only some individual steps of the attack are detected by detection systems. Based on the results, we use both basic features and context features in the detection framework.

We further employ the SHAP algorithm (Lundberg and Lee, 2017) to analyze the importance of specific features in the *lldos*, *iscx*, and CPTC datasets. The feature importance of CPTC is calculated by the mean value of 3 CPTC datasets. Fig. 7 shows the importance of 11 features. The inner circle shows the overall comparison of two feature categories, while the outer circle shows the proportion of each feature in a counterclockwise direction. The blue color represents basic features, while the green color represents contextual features. Both context and basic features have a significant impact on the final result, and the sum importance of context-based features for each dataset is 0.65, 0.54, and 0.50, respectively, which is higher than that of basic features. For the *lldos* and *iscx* datasets, the features that contribute the most to the detection are *Feature 10: # Alert of each attack stage in the sequence* and *Feature 4: Attack stages*. The importance of Frequency Encoding based features, such as *Alert subject* and *Alert signature*, might be falsely in-

⁶ <https://stable-baselines.readthedocs.io/en/master/>.

⁷ <https://nvd.nist.gov/vuln/detail/CVE-2001-0341>.

Table 4

Summary of features contribution to initial alert ranking in anomaly detection. The Precision, Recall, and F-score improvement implies that employing context features yields better performances than using only basic features.

Datasets	Features*	Precision	Precision improvement	Recall	Recall improvement	F1-score	F1-score improvement
lldos	BF	0.4711	-	0.9078	-	0.6188	-
	CF	0.8594	0.3883↑	0.8250	-0.0828↓	0.8370	0.2182↑
	BF+CF	0.9124	0.4413↑	0.7375	-0.1703↓	0.8151	0.1963↑
iscx	BF	0.8615	-	0.4341	-	0.5645	-
	CF	0.9968	0.1353↑	0.7317	0.2976↑	0.8451	0.2806↑
	BF+CF	0.9966	0.1351↑	0.7329	0.2988↑	0.8445	0.2800↑
CPTC-T1	BF	0.2450	-	0.9845	-	0.3924	-
	CF	0.3343	0.0893↑	0.9932	0.0087↑	0.5002	0.1078↑
	BF+CF	0.3978	0.1528↑	0.9901	0.0056↑	0.5675	0.1748↑
CPTC-T5	BF	0.3593	-	0.9752	-	0.5252	-
	CF	0.5198	0.1605↑	0.9948	0.0196↑	0.6828	0.1576↑
	BF+CF	0.5340	0.1747↑	0.9876	0.0124↑	0.6932	0.1680↑
CPTC-T8	BF	0.3981	-	0.9565	-	0.5622	-
	CF	0.4710	0.0729↑	0.9947	0.0382↑	0.6393	0.0771↑
	BF+CF	0.5173	0.1192↑	0.9860	0.0295↑	0.6785	0.1163↑

* BF: Basic Features, CF: Context Features, BF + CF: Basic Features and Context Features

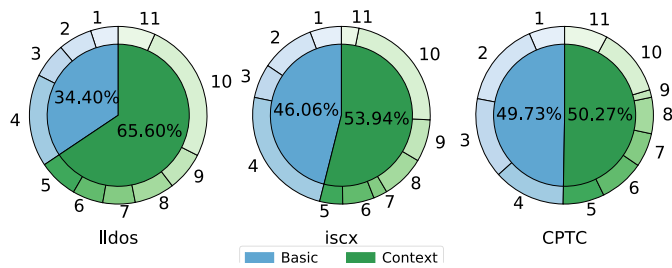


Fig. 7. Feature importance of lldos, iscx and CPTC datasets. The inner circle represents the overall weight of two categories of features. The blue color denotes basic features, and the green color corresponds to context features. The outer circle illustrates the weight of each feature, arranged in a counterclockwise order following the basic and context features listed in Table 2. Both context and basic features have a significant impact on the final result, and the sum importance of context features for each dataset is 0.65, 0.54, and 0.50, respectively, which is higher than that of basic features.

flated in some cases because the alert distribution in the background data and the original CPTC dataset may be inconsistent.

5.5. Active learning improvement (RQ2)

In the anomaly detection experiment, we chose the optimal threshold based on the ROC curve to achieve the best detection results. However, in practical scenarios, adjusting the threshold manually can be challenging. Additionally, as shown in Table 4, the Precision of the three feature selection methods is low, especially on CPTC datasets, indicating that only relying on anomaly detection algorithms may not detect all real attacks. Fig. 8 illustrates a 2-D visualization of the CPTC-T1 dataset using *t*-SNE⁸, where four types of nodes are present: TN, TP, FP, and FN, which are identified by the anomaly detection algorithms. We split anomaly and normal results based on the *Budget*. Specifically, using the anomaly scores, the top *Budget* samples are labeled as anomalies (positive instances), while the remaining samples are labeled as normal (negative instances). As shown in Fig. 8, the distribution of attack alerts is widely dispersed, and the anomaly detection algorithm is only able to detect a small portion of the true attack alerts. Moreover,

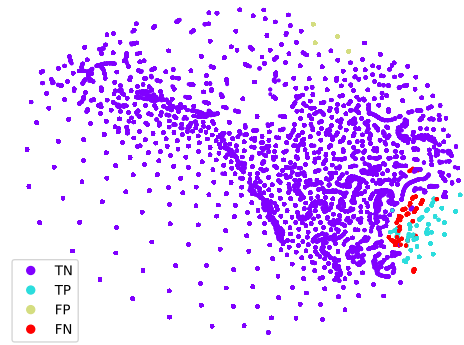


Fig. 8. Alert distribution of CPTC-T1 using *t*-SNE. The distribution of attack alerts is scattered, and the anomaly detection algorithm can only detect a small portion of the true attack alerts.

in the top *Budget* samples, there are numerous false positives, indicating that the non-attack alerts are ranked higher than real attack alerts. The occurrence of false high-risk alerts also causes real alerts to rank lower, subsequently reducing their chances of being thoroughly analyzed. This observation highlights the importance of integrating an active learning model to enhance the overall detection capability.

To enhance the precision and recall of anomaly detection, we have integrated active learning into the alert re-ranking process. This allows AlertPro to interact with experts, adjusting the ranking of alerts and identifying more potential attack alerts. During each dataset evaluation, we train a model using the four remaining datasets. The impact of training dataset selection is explored in detail in Section 5.6.

We compare AlertPro with two other algorithms that also apply expert feedback to detect anomalies: an online update algorithm AAD (Das et al., 2016; Das, 2018) and a reinforcement learning algorithm Meta-AAD (Zha et al., 2020). Both AAD and Meta-AAD are general models of active learning. For the baseline anomaly detection approach, we use the classic *Isolation Forest* algorithm and select *Budget* alerts with the highest rank for the analyst. All three algorithms use alert features extracted in section 4 as input. In this experiment, we set the AAD parameters as *detector type* = *Isolation Forest*, *query confidence* = 0.03, and we use default settings for Meta-AAD as *learning rate* = 2.5×10^{-4} , *gamma* = 0.95, rollout steps $T = 128$, *entropy coefficient* = 0.01.

In this experiment, we evaluate AlertPro using three commonly used reinforcement learning algorithms: D3QN, A2C, and PPO2, with de-

⁸ <https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>.

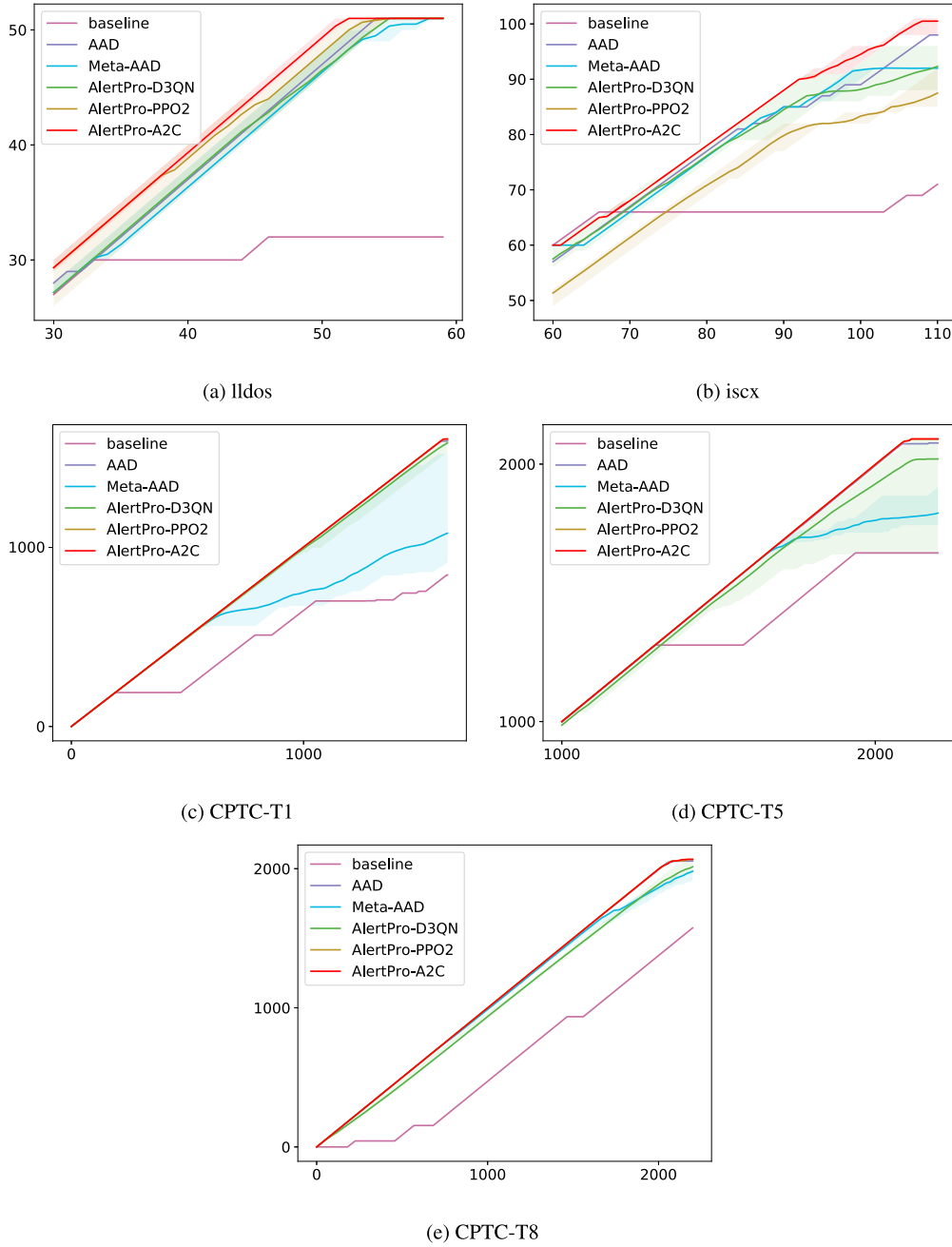


Fig. 9. Anomaly Discovery Curve of AlertPro. The horizontal axis represents the number of interactions, and the vertical axis represents the number of real alerts (TP) found. AlertPro outperformed both Meta-AAD and AAD in terms of the improvement in detection rate achieved through active learning.

fault parameters provided by Stable-Baselines, a popular platform for training and evaluating reinforcement learning models. The *budget* is set to the nearest number of real attack alerts in the dataset as shown in Table 5, which are: 55, 110, 1610, 2100, and 2070, respectively. We evaluate each dataset 10 times and take the average result.

Fig. 9 shows the Anomaly Discovery Curve of the alert active learning method. It indicates that active learning is effective in improving detection metrics compared to the baseline (represented by the pink line), which uses only the anomaly detection algorithm, across all five datasets. The most significant improvement is observed in the CPTC datasets. This is due to the low precision of anomaly detection, which makes it difficult to detect actual attack alerts without feedback.

At the beginning of the interactions, the baseline method only identifies a few or no alerts, while the active learning method efficiently identifies alerts through continuous feedback. Furthermore, the num-

ber of alerts found by the active learning method increases steadily, while the baseline method experiences prolonged stagnation periods. For instance, in the CPTC-T1 dataset, the baseline method shows three long platform periods during the interaction process, indicating the existence of noise alerts with high scores. The anomaly detection algorithm constantly forwards high-score alerts to experts, without paying attention to whether they are truly correct. In contrast, the active learning method avoids such stagnation periods, and every alert found is a legitimate attack alert. The results demonstrate that the active learning approach is capable of identifying real attack alerts. Through the interactive process involving analysts, the method leverages expert knowledge, continuously re-ranks alerts, and gradually uncovers high-risk alerts. Overall, active learning proves to be a powerful technique for alert analysis.

Table 5

Comparison of the Cumulative Precision (cP) and Cumulative Recall (cR) for baselines and active learning models. Column *Improvement* means improvement of AlertPro-A2C over baseline.

	baseline		AAD		Meta-AAD		AlertPro-D3QN		AlertPro-PPO2		AlertPro-A2C		Improvement	
	cP	cR	cP	cR	cP	cR	cP	cR	cP	cR	cP	cR	cP	cR
lldos	0.5818	0.6275	0.9273	1.0000	0.9182	0.9902	0.9273	1.0000	0.9273	1.0000	0.9273	1.0000	0.3455	0.3725
iscx	0.6455	0.6636	0.8909	0.9159	0.8476	0.8714	0.6727	0.6916	0.7103	0.7302	0.9157	0.9414	0.2702	0.2778
CPTC-T1	0.5210	0.5245	0.9821	0.9888	0.6645	0.6690	0.9753	0.9820	0.9885	0.9952	0.9883	0.9950	0.4673	0.4705
CPTC-T5	0.7855	0.7862	0.9715	0.9724	0.8537	0.8545	0.9537	0.9546	0.9952	0.9962	0.9952	0.9962	0.2097	0.2099
CPTC-T8	0.6986	0.6996	0.9894	0.9908	0.9212	0.9225	0.9366	0.9380	0.9908	0.9923	0.9895	0.9909	0.2909	0.2913
average	0.6465	0.6603	0.9522	0.9736	0.8410	0.8615	0.8931	0.9132	0.9224	0.9428	0.9632	0.9847	0.3167	0.3244

Moreover, we observed that the improvement in detection rate varied across the different active learning methods. Specifically, AlertPro outperformed both Meta-AAD and AAD in terms of the improvement in detection rate achieved through active learning. The performance of AAD (represented by the purple line) is comparable to that of AlertPro. However, AlertPro-A2C (represented by the red line) slightly exceeds AAD on the *iscx*, CPTC-T1, and CPTC-T5 datasets. Moreover, AlertPro can extract hidden features from analyst feedback, which can provide explainability for the decision-making process. While the AAD method is a black box model, making it difficult for analysts to understand why certain alerts are classified as high-risk, thus reducing the model's trust. The explainability of AlertPro is discussed in Section 6. The three algorithms of AlertPro exhibit similar performance and are nearly overlapping in Fig. 9. This result implies that each reinforcement learning method can effectively detect alerts, and it also validates the state and reward design of AlertPro.

In order to quantitatively analyze the impact of feedback, Table 5 provides the Cumulative Precision (cP) and Cumulative Recall (cR) of each method. In Experiment 5.4, the IF algorithm predicts alerts based on the optimal threshold, while in Table 5, the baseline identifies alerts in the top *Budget* as true alerts. Therefore, the lower recall in Table 5 indicates that there are fewer true alerts among the top *Budget* alerts. The active learning method achieves an average improvement of 31.67% in cP and 32.44% in cR over the baseline. After the feedback process, the model was able to achieve 96.32% cP and 98.47% cR on average, indicating that experts were able to identify more alerts with the assistance of the model by focusing only on real alerts. AlertPro was able to identify the maximum number of real alerts in all 5 datasets. Among the three reinforcement learning models used in AlertPro, A2C performed the best in the CPTC-T1 dataset, reaching a maximum of 99.52% CP and 99.62% cR. It is worth noting that AlertPro-PPO2 utilizes the same model parameters as Meta-AAD, but outperforms Meta-AAD in most datasets, especially CPTC-T5. This observation indicates that the inclusion of historical features and customized reward calculation in AlertPro enhances its ability to detect attack alerts.

5.6. Availability and practicality (RQ3)

Model availability refers to the performance of the model during both the training and detection phases. In order to effectively analyze alerts, it is crucial to periodically retrain the model to keep up with changes in the network environment and attack methods. Moreover, the quality of the data used to train the model affects its ability to transfer to a new environment. The time taken for training and using the model is also an important factor in its convenience. To answer RQ3, we propose two evaluation metrics: (1) **Training dataset selection**, and (2) **Model training and detection time**.

5.6.1. Training dataset selection

For training datasets, the model should not strictly necessary to use labeled alert datasets from the same network environment or attack scenario. This is because the model heavily relies on constant feedback from experts during training, allowing it to adapt to different

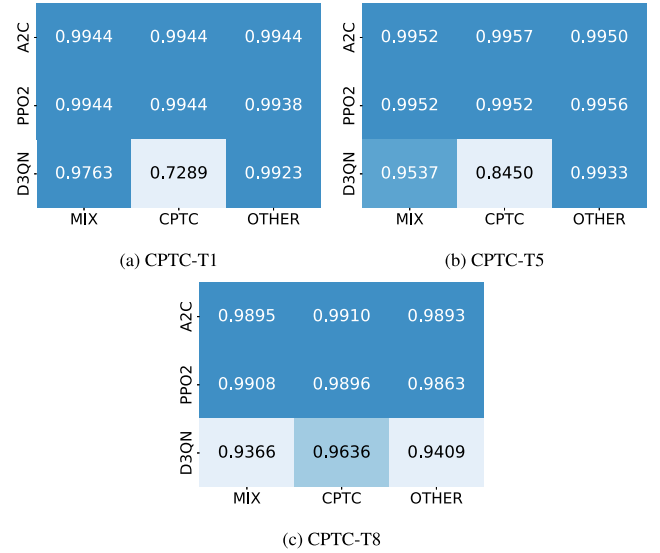


Fig. 10. Heat map of evaluation on three CPTC datasets with AlertPro training using different types of datasets. The horizontal axis represents the datasets used for training, which include (1) a **MIX** of four other datasets, (2) two other **CPTC** datasets from the same network, and (3) **OTHER** two unrelated datasets: *lldos* and *iscx*. The vertical axis represents different models. The numbers in the figure represent the Cumulative Precision of training the model on the dataset corresponding to the horizontal axis and subsequently testing it on the dataset corresponding to the subgraph title. For instance, in Subfigure (a), the cP values of three models tested on the CPTC-T1 dataset are shown. These models were respectively trained on the MIX dataset, the merged CPTC-T5 and CPTC-T8 datasets, and the merged *iscx* and *lldos* datasets.

environments and attack methods. Therefore, any labeled dataset with alerts can be used for training, even if it is from a different network environment. However, using a variety of datasets for training can better discover the diversity of alerts, leading to better performance of the model. We conducted experiments to evaluate the impact of different training datasets on AlertPro's model availability. Specifically, we trained AlertPro using three types of training datasets: (1) a MIX of other four datasets; (2) two other CPTC datasets which come from the same network; (3) OTHER two unrelated datasets: *lldos* and *iscx*. As shown in Fig. 10, the cP of training with the above three types of datasets is quite similar. All results of A2C and PPO2 on the three training datasets are dark-blue, indicating that the model can achieve the best results regardless of which dataset is used for training. This observation supports our assumption that any labeled dataset can be used for training. For the D3QN method in the CPTC-T1 and CPTC-T5 datasets, using mixed datasets yielded better results than using only the CPTC dataset, and even using unrelated datasets can get the best results. This is possibly due to the limited data types in the training dataset, leading to insufficient exploration of states during the training of D3QN. In the case of the CPTC-T8 dataset, training with irrelevant datasets still yields

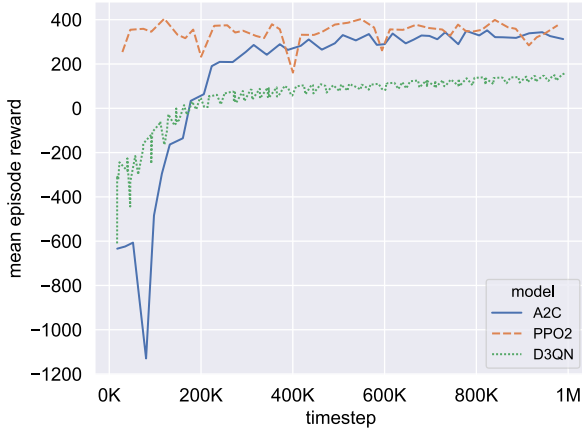


Fig. 11. Mean episode reward of models in 1M timesteps. It is evident that the episode reward can be stabilized with a few training steps, and all models become flattened after approximately 40 K timesteps.

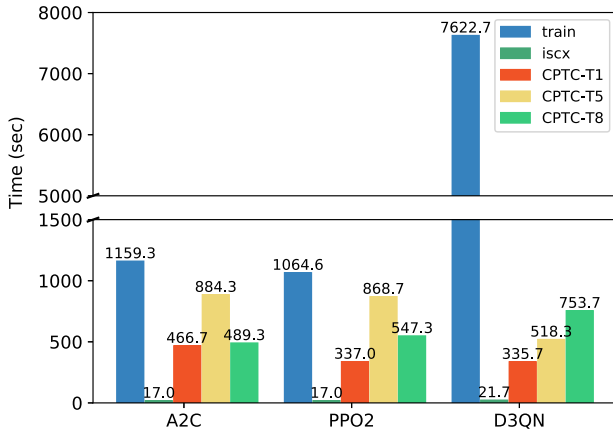


Fig. 12. Time of the training phase and detection phase. The horizontal axis is three algorithms. For each algorithm, the first column (blue) is training time, and the rest columns are the detection time on different datasets.

favorable results, with only a marginal difference of 0.0236 compared to training with the CPTC dataset.

5.6.2. Model training and detection time

We also evaluate the availability of the model by assessing the time required for both the training and detection phases. Although AlertPro does not require retraining, it can be efficiently retrained when necessary, such as for feature adjustments or additions. The training time of the model is a critical factor in its availability, as longer training time makes retraining less convenient. To evaluate this, we analyzed the relationship between the mean episode reward and training steps, as shown in Fig. 11. The results demonstrate that episode reward can be stabilized with a few training steps, and all models become flattened after 40 K timesteps.

Fig. 12 shows the time consumption in model training and detection phases. For evaluation purposes, we excluded the *lidos* dataset due to its limited number of samples for time evaluation. For each algorithm, the first column (blue) represents training time. We trained the models for 1M timesteps, and both A2C and PPO2 are completed in approximately 1000 seconds. Based on the stability of the episode reward, it merely takes about 200 seconds to reach a steady state of 40 K timesteps. However, the D3QN model takes much longer to train, with an average training time of 7622.7 seconds, making it less suitable for scenarios requiring shorter training time. The rest columns of each algorithm are the detection time in different datasets. In comparison, the

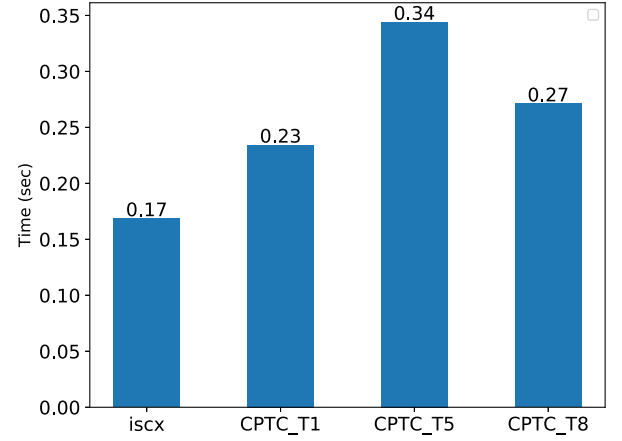


Fig. 13. Time of one interaction. The processing time of AlertPro for completing a rank is relatively short and within 0.5 seconds, even for larger size and more *Budget* datasets like CPTC-T5.

detection phase takes less time than training and the time consumption across models is similar.

Fig. 13 shows the average processing time per alert. The time consumption of the detection phase depends on the dataset size and interaction *Budget*. As the dataset size increases, the model needs more time to process the alerts and calculate the Euclidean distance. In each interaction, the model must re-rank all the alerts, which also contributes to the processing time. The processing time of the model for re-ranking and selecting high-risk alerts remains relatively short, typically within 0.5 seconds, even for larger size and more *Budget* datasets like CPTC-T5. This indicates that the model can effectively select alerts from the pool in real-time scenarios. Furthermore, the model's processing time is much faster than the time it takes for humans to manually assess whether an alert is a real attack. This highlights the potential of using AI-based alert selection systems to assist security analysts in their tasks and improve the overall efficiency of alert analysis.

6. Discussion

In this section, we discuss four aspects of the AlertPro framework that require attention. We begin by discussing the explainability of AlertPro in Section 6.1, and then we point out that AlertPro can be used in an enterprise environment in Section 6.2 and provide the necessary context for its deployment. Subsequently, we turn our attention to the limitations of AlertPro in Section 6.3, and finally, we point out future improvements in Section 6.4.

6.1. Explainability

For analysts, it is essential that the output of AlertPro is explainable, enabling them to comprehend the reasons behind certain alerts being predicted as high-risk. The explainability of AlertPro is mainly reflected in two aspects: (1) Feature importance. As shown in Fig. 7, two features about attack stages make great contributions to anomaly detection. These two features are *Feature 4: Attack stages* and *Feature 10: # Alerts of each attack stage in the sequence*. The attack stage is a concept unique to multi-step attacks, which is consistent with AlertPro's design motivation for multi-step attack detection. (2) Decision path. AlertPro first uses isolated forests to calculate the anomaly detection score and then uses active learning to correct the anomaly detection bias. If an alert has a low score in anomaly detection but is predicted as a potential high-risk alert by the active learning model, and is verified as an attack alert by the analyst during alert validation, it means that this alert is strongly correlated with some alerts that have been verified as attack alerts by analysts. This strong correlation can be reflected in historical features.

6.2. Applicability in enterprises

Although AlertPro has not been used in an actual enterprise network environment, we have thoroughly considered this issue during the design, experimentation, and evaluation of AlertPro. The applicability of AlertPro in enterprise settings can be observed in the following three aspects: (1) Dataset. Similar to Fu et al. (2021), we simulated the enterprise network using merged datasets, as detailed in Section 5.1. The CPTC dataset itself is generated by simulating penetration testing in an automobile enterprise network. Moreover, we used snort to generate noise alerts in the background traffic, i.e., false positives and irrelevant alerts, which makes CPTC more representative of a real-world scenario. (2) Evaluation. We conducted experiments on training dataset selection, and model training and detection time in Section 5.6. These evaluations underscore AlertPro's potential for integration into real-world enterprise network environments, considering both its performance and overhead. (3) Case study. In addition, our case study, detailed in Section 5.3, demonstrated AlertPro's ability to unveil previously undisclosed attacks hidden within the public dataset iscx, illustrating its capacity to enhance security posture beyond existing tools.

AlertPro is designed to improve the work efficiency of alert analysts, more specifically, alert triage specialists. As an alert prioritization framework, AlertPro only fetches the potential highest alert when queried by alert triage specialists, allowing them to allocate their efforts effectively. AlertPro can be used in the SOC where a large volume of alerts is generated, and alert triage specialists face the alert fatigue problem.

To effectively deploy AlertPro within SOC environments, several key elements must be considered: (1) Alert collection and standardization: In an enterprise, alerts may originate from various security products or sensors, such as IDS, firewalls, network monitoring tools, and more. These alerts come in different formats. As a first step, these alerts need to be aggregated and standardized into a unified format, ensuring that they can be effectively processed by AlertPro. (2) User Interface (UI) for expert analysis: A user-friendly UI interface is essential to support security experts in their analysis. This interface should empower experts to review, validate, and annotate the alerts from AlertPro. The UI serves as a conduit for experts to provide their insights, ultimately offering valuable feedback on the alert prioritization. Additionally, it's necessary to include various interfaces to match the existing system with AlertPro, such as interfaces for storing pre-processed alerts and assessed alerts in a database.

6.3. Limitations

Upon the alert validation process outlined in Section 2.4 and the design overview of AlertPro presented in Section 4.1, there are two limitations: (1) Feedback information. The feedback provided by experts encompasses decisions, labels, severity assessment, and disposition priority. However, AlertPro currently only utilizes the decision aspect of this feedback. It is worth revisiting the possibility of incorporating additional aspects of this feedback into AlertPro. (2) In AlertPro, analysts receive only one high-risk alert for each query. However, if multiple items are returned simultaneously, the efficiency of the alert validation process might be improved. For instance, by enabling AlertPro to return top N high-risk alerts at once, analysts can validate and process alerts more efficiently. Hence, upgrading AlertPro to a batch-processing mode would be beneficial.

Moreover, AlertPro operates under the assumption that alert analysts are experienced experts and that alert analysts' feedback is reliable. This assumption forms the basis for utilizing expert feedback within AlertPro to rectify any bias present in the machine-learning model.

6.4. Future work

The future work is mainly to integrate more expert feedback, provide analysts with more disposal suggestions, and apply batch-processing technologies to improve the work efficiency of the entire SOC. (1) Incorporating richer feedback information. As introduced in Section 2.4 analysts often provide detailed feedback during the alert validation process, which includes labels such as known attack tool scanning, false positives, normal logs, and ignorable alerts. They also provide reasons for their identification, such as IoC-related IPs and attack payloads. Utilizing this richer feedback information can involve adjusting the feature weights to improve the accuracy and interpretability of the model. Exploring how to effectively utilize this feedback is a promising research direction for the future. (2) Enriching disposal suggestions. The current alert assessment of AlertPro provides binary results (0 or 1) indicating whether an alert is classified as an attack or non-attack. However, analysts may have other decisions regarding alerts, such as ignoring them, blocking the IP, or continuously tracking them for further analysis. Developing automated methods to suggest these actions for different types of alerts is another important research direction for future studies. (3) Combining rule-based models to process alerts in batches. In practical scenarios, analysts often employ batch operations to efficiently manage alerts during urgent attack situations. This involves implementing rules such as IP blocking and signature selection. By combining AlertPro with rule-based models, analysts can effectively filter alerts. To illustrate, after alerts are identified by AlertPro, analysts can label them using predefined rules. For example, if an attacking IP is verified to be targeting the enterprise network based on AlertPro, all alerts associated with that IP can be classified as attack alerts through a straightforward rule-matching process. This combined approach enables analysts to streamline their workflow and prioritize their focus on high-priority alerts identified by AlertPro while leveraging rule-based models to handle other types of alerts efficiently. Of course, implementing the above ideas will inevitably increase the technical complexity of the model, which is still essentially a trade-off problem.

7. Conclusion

This paper introduces AlertPro, a self-evolving framework aimed at combating alert fatigue and improving the efficiency of the entire SOC workflow. Through extensive experiments on five real-world datasets, it has been demonstrated that AlertPro effectively utilizes context features to enhance the accuracy of multi-step attack detection in anomaly detection. Additionally, AlertPro successfully filters out low-risk alerts and highlights high-risk ones by extracting history features from expert feedback and taking advantage of an active learning model to incorporate expert knowledge. However, it is important to acknowledge the limitations. AlertPro currently only utilizes the decision aspect of expert feedback and returns only one high-risk alert for each query. While, feedback provided by experts encompasses decisions, labels, severity assessment, and disposition priority. The future work primarily involves integrating additional expert feedback, offering analysts more disposal suggestions, and implementing batch-processing technologies to enhance the overall work efficiency of the SOC.

Overall, AlertPro makes a valuable contribution to combating alert fatigue and improving the detection rate of multi-step attacks. The findings presented in this paper provide valuable insights for the development and deployment of practical solutions in real-world scenarios.

CRedit authorship contribution statement

Xiaoyu Wang: Investigation, Methodology, Writing – Reviewing & Editing, Draft Preparation. **Xiaobo Yang:** Methodology, Writing – Reviewing & Editing, Draft Preparation. **Xueping Liang:** Supervision,

Writing – Reviewing & Editing, Draft Preparation. **Xiu Zhang:** Methodology, Writing – Reviewing & Editing, Draft Preparation, Supervision. **Wei Zhang:** Investigation, Writing – Reviewing & Editing. **Xiaorui Gong:** Methodology, Reviewing – Editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgement

This work is supported by the Strategic Priority Research Program of the Chinese Academy of Sciences (No. XDC02040100), the Key Laboratory of Network Assessment Technology at the Chinese Academy of Sciences, and the Beijing Key Laboratory of Network Security and Protection Technology.

References

- Al-Saedi, Karim, Ramadass, Sureswaran, Almomani, Dr. Ammar, Manickam, Selvakumar, 2012. Collection mechanism and reduction of ids alert. *Int. J. Comput. Appl.* 58, 11.
- Alahmadi, Bushra A., Axon, Louise, Martinovic, Ivan, 2022. 99% false positives: a qualitative study of SOC analysts' perspectives on security alarms. In: 31st USENIX Security Symposium (USENIX Security 22). USENIX Association, Boston, MA, pp. 2783–2800.
- Aminanto, Muhammad Erza, Zhu, Lei, Ban, Tao, Isawa, Ryoichi, Takahashi, Takeshi, Inoue, Daisuke, 2019. Combating threat-alert fatigue with online anomaly detection using isolation forest. In: *International Conference on Neural Information Processing*. Springer, pp. 756–765.
- Ban, Tao, Samuel, Ndichu, Takahashi, Takeshi, Inoue, Daisuke, 2021. Combat security alert fatigue with ai-assisted techniques. In: *Cyber Security Experimentation and Test Workshop*, pp. 9–16.
- Das, Shubhomoy, 2018. Active anomaly discovery. https://github.com/shubhomoydas/ad_examples. (Accessed 19 March 2023). Online.
- Das, Shubhomoy, Wong, Weng-Keen, Dietterich, Thomas, Fern, Alan, Emmott, Andrew, 2016. Incorporating expert feedback into active anomaly discovery. In: 2016 IEEE 16th International Conference on Data Mining (ICDM). IEEE, pp. 853–858.
- Ding, Kaize, Li, Jundong, Liu, Huan, 2019. Interactive anomaly detection on attributed networks. In: *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining*, pp. 357–365.
- FireEye, 2015. The numbers game: How many alerts are too many to handle. [EB/OL]. (Accessed 6 March 2021). Online.
- Fu, Chuanpu, Li, Qi, Shen, Meng, Xu, Ke, 2021. Realtime robust malicious traffic detection via frequency domain analysis. In: *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pp. 3431–3446.
- Haas, Steffen, Fischer, Mathias, 2018. Gac: graph-based alert correlation for the detection of distributed multi-step attacks. In: *Proceedings of the 33rd Annual ACM Symposium on Applied Computing*, pp. 979–988.
- Hassan, Wajih Ul, Guo, Shengjian, Li, Ding, Chen, Zhengzhang, Jee, Kangkook, Li, Zhichun, Bates, Adam, 2019. Nodose: combatting threat alert fatigue with automated provenance triage. In: *Network and Distributed Systems Security Symposium*.
- Husák, Martin, Kašpar, Jaroslav, 2019. Aida framework: real-time correlation and prediction of intrusion detection alerts. In: *Proceedings of the 14th International Conference on Availability, Reliability and Security*, pp. 1–8.
- Hutchins, Eric M., Cloppert, Michael J., Amin, Rohan M., et al., 2011. Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. *Leading Issues in Information Warfare & Security Research* 1 (1), 80.
- Lamba, Hemank, Akoglu, Leman, 2019. Learning on-the-job to re-rank anomalies from top-1 feedback. In: *Proceedings of the 2019 SIAM International Conference on Data Mining*. SIAM, pp. 612–620.
- Liu, Fei Tony, Ting, Kai Ming, Zhou, Zhi-Hua, 2008. Isolation forest. In: *Eighth IEEE International Conference on Data Mining*. IEEE, pp. 413–422.
- Liu, Jia, Zhang, Runzi, Liu, Wenmao, Zhang, Yinghua, Gu, Dujuan, Tong, Mingkai, Wang, Xingkai, Xue, Jianxin, Wang, Huanran, 2022a. Context2vector: accelerating security event triage via context representation learning. *Inf. Softw. Technol.* 146, 106856.
- Liu, Yushan, Shu, Xiaokui, Sun, Yixin, Jang, Jiyong, Mittal, Prateek, 2022b. Rapid: real-time alert investigation with context-aware prioritization for efficient threat discovery. In: *Proceedings of the 38th Annual Computer Security Applications Conference*, pp. 827–840.
- Lundberg, Scott M., Lee, Su-In, 2017. A unified approach to interpreting model predictions. In: Guyon, I., Von Luxburg, U., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (Eds.), *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc.
- Lyu, Minzhao, Gharakheili, Hassan Habibi, Russell, Craig, Sivaraman, Vijay, 2022. Enterprise dns asset mapping and cyber-health tracking via passive traffic analysis. *IEEE Trans. Netw. Serv. Manag.*
- MIT Lincoln Lab, 2000. 2000 darpa intrusion detection scenario specific datasets. <https://www.ll.mit.edu/r-d/datasets/2000-darpa-intrusion-detection-scenario-specific-datasets>. (Accessed 10 April 2023). Online.
- Mnih, Volodymyr, Badia, Adria Puigdomenech, Mirza, Mehdi, Graves, Alex, Lillicrap, Timothy, Harley, Tim, Silver, David, Kavukcuoglu, Koray, 2016. Asynchronous methods for deep reinforcement learning. In: *International Conference on Machine Learning*. PMLR, pp. 1928–1937.
- Munaiah, Nuthan, Rahman, Akond, Pelletier, Justin, Williams, Laurie, Meneely, Andrew, 2019. Characterizing attacker behavior in a cybersecurity penetration testing competition. In: 2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM). IEEE, pp. 1–6.
- Nadeem, Azqa, Verwer, Sicco, Moskal, Stephen, Yang, Shanchieh Jay, 2021. Alert-driven attack graph generation using s-pdf. *IEEE Trans. Dependable Secure Comput.* 19 (2), 731–746.
- Navarro, Julio, Deruyver, Aline, Parrend, Pierre, 2018a. A systematic survey on multi-step attack detection. *Comput. Secur.* 76, 214–249.
- Navarro, Julio, Deruyver, Aline, Parrend, Pierre, 2018b. A systematic survey on multi-step attack detection. *Comput. Secur.* 76, 214–249.
- Ning, Peng, Cui, Yun, Reeves, Douglas S., 2002. Constructing attack scenarios through correlation of intrusion alerts. In: *Proceedings of the 9th ACM Conference on Computer and Communications Security*, pp. 245–254.
- Nisioti, Antonia, Mylonas, Alexios, Yoo, Paul D., Katos, Vasilios, 2018. From intrusion detection to attacker attribution: a comprehensive survey of unsupervised methods. *IEEE Commun. Surv. Tutor.* 20 (4), 3369–3388.
- Njogu, Humphrey, Jiawei, Luo, 2010. Using Alert Cluster to Reduce Ids Alerts, pp. 467–471.
- Ponemon, 2015. The cost of malware containment. (Accessed 11 April 2023). Online.
- Powers, David M.W., 2020. Evaluation: from precision, recall and f-measure to roc, informedness, markedness and correlation. Preprint arXiv:2010.16061.
- Riley, Michael, Elgin, Ben, Lawrence, Dune, Matlack, Carol, 2014. Missed alarms and 40 million stolen credit card numbers: how target blew it. <https://www.bloomberg.com/news/articles/2014-03-13/target-missed-warnings-in-epic-hack-of-credit-card-data>. (Accessed 10 April 2023). Online.
- Schulman, John, Wolski, Filip, Dhariwal, Prafulla, Radford, Alec, Klimov, Oleg, 2017. Proximal policy optimization algorithms. Preprint arXiv:1707.06347.
- Shiravi, Ali, Shiravi, Hadi, Tavallaee, Mahbod, Ghorbani, Ali A., 2012. Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *Comput. Secur.* 31 (3), 357–374.
- Shittu, Riyanat, Healing, Alex, Ghanee-Hercock, Robert, Bloomfield, Robin, Outmet, Rajarajan Muttukrishnan, 2014. A new metric for prioritising intrusion alerts using correlation and outlier analysis. In: 39th Annual IEEE Conference on Local Computer Networks. IEEE, pp. 322–330.
- Siddiqui, Md Amran, Fern, Alan, Dietterich, Thomas G., Wright, Ryan, Theriault, Alec, Archer, David W., 2018a. Feedback-guided anomaly discovery via online optimization. In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2200–2209.
- Siddiqui, Md Amran, Fern, Alan, Dietterich, Thomas G., Wright, Ryan, Theriault, Alec, Archer, David W., 2018b. Feedback-guided anomaly discovery via online optimization. In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2200–2209.
- Siddiqui, Md Amran, Stokes, Jack W., Seifert, Christian, Argyle, Evan, McCann, Robert, Neil, Joshua, Carroll, Justin, 2019. Detecting cyber attacks using anomaly detection with explanations and expert feedback. In: *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, pp. 2872–2876.
- van Ede, Thijs, Aghakhani, Hojjat, Spahn, Noah, Bortolameotti, Riccardo, Cova, Marco, Continella, Andrea, van Steen, Maarten, Peter, Andreas, Kruegel, Christopher, Vigna, Giovanni, 2022. Deepcase: semi-supervised contextual analysis of security events. *IEEE Secur. Priv.*
- Van Hasselt, Hado, Guez, Arthur, Silver, David, 2016. Deep reinforcement learning with double q-learning. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30.
- Veeramachaneni, Kalyan, Arnaldo, Ignacio, Korrapati, Vamsi, Bassias, Constantinos, Li, Ke, 2016. Ai²: training a big data machine to defend. In: 2016 IEEE 2nd International Conference on Big Data Security on Cloud (BigDataSecurity), IEEE International Conference on High Performance and Smart Computing (HPSC), and IEEE International Conference on Intelligent Data and Security (IDS). IEEE, pp. 49–54.
- Wang, Meng, Lu, Yiqin, Qin, Jiancheng, 2020. A dynamic mlp-based ddos attack detection method using feature selection and feedback. *Comput. Secur.* 88, 101645.
- Wang, Ziyu, Schaul, Tom, Hessel, Matteo, Hasselt, Hado, Lanctot, Marc, Freitas, Nando, 2016. Dueling network architectures for deep reinforcement learning. In: *International Conference on Machine Learning*. PMLR, pp. 1995–2003.

- Yu, Hualong, Mu, Chaoxu, Sun, Changyin, Yang, Wankou, Yang, Xibei, Zuo, Xin, 2015. Support vector machine-based optimized decision threshold adjustment strategy for classifying imbalanced data. *Knowl.-Based Syst.* 76, 67–78.
- Zha, Daochen, Lai, Kwei-Herng, Wan, Mingyang, Hu, Xia, 2020. Meta-aad: active anomaly detection with deep reinforcement learning. In: 2020 IEEE International Conference on Data Mining (ICDM). IEEE, pp. 771–780.
- Zhang, Haotian, Ma, Xu, 2022. Misleading attention and classification: an adversarial attack to fool object detection models in the real world. *Comput. Secur.* 122, 102876.
- Zhou, Zhi-Hua, Liu, Xu-Ying, 2005. Training cost-sensitive neural networks with methods addressing the class imbalance problem. *IEEE Trans. Knowl. Data Eng.* 18 (1), 63–77.

Xiaoyu Wang is a Ph.D student at University of Chinese Academy of Sciences, majoring in Cyber Security. She received her B.S. degree in Information Security from Nankai University in 2017. Her research interests include intrusion detection, alert analyze and attack investigation. (Email: wangxiaoyu@iie.ac.cn).

Xiaobo Yang received her Ph.D. degree in Information and Communication Engineering from Beijing Institute of Technology in 2020. She currently works at the Institute of Information Engineering, Chinese Academy of Sciences as an engineer. Her research interests include network security, machine learning applications, and offensive and defensive technologies. (Email: yangxiaobo@iie.ac.cn).

Xueping (Susan) Liang is an Assistant Professor in the Department of Information Systems and Business Analytics in Florida International University. She received her Ph.D. from University of Chinese Academy of Sciences in 2019, and B.S. from Beijing Institute of Technology in 2013. Her research is centered around cybersecurity, Blockchain,

data provenance mechanisms, privacy protection, healthcare and Internet of Things (IoT). Specifically, she is interested in distributed consensus models in Blockchain technology, cyber-resiliency, and various practical issues in cloud computing security. She has published more than 30 conference and journals papers, book chapters at reputed venues. One of her papers has been awarded as “Top 50 Influential Papers in Blockchain” at Blockchain Connect Conference, San Francisco, in January 2019. Her research has been funded by the Office of Naval Research. (Email: liangxueping2015@gmail.com).

Xiu Zhang received her Ph.D. degree in Information Security from University of Chinese Academy of Sciences in 2021. She works at the Institute of Information Engineering, Chinese Academy of Sciences as an engineer. Her research interests focus on penetration testing, network attack and defense. (Email: zhangxiu@iie.ac.cn).

Wei Zhang received her B.S. degree in Control Science and Engineering with a focus in Artificial Intelligence Application in 2020 at Beijing Institute of Technology. She works at Institute of Information Engineering, Chinese Academy of Sciences as an engineer on system development of attack monitoring. Her search interests include machine learning applications in cybersecurity and system development. (Email: zhangwei4044@iie.ac.cn).

Xiaorui Gong received a B.S. degree from Peking University in 2014. He is a professor at School of Cyber Security, University of Chinese Academy of Sciences. He works as a senior engineer at the Institute of Information Engineering, Chinese Academy of Sciences. His research focuses on the offensive and defensive technology in cybersecurity. (Email: gongxiaorui@iie.ac.cn).